🧠 AI Evolution Program

Parte 11 — IA encarnada, robótica y uso de computadores

Lleva la toma de decisiones al mundo físico y a interfaces digitales, con sensores, control, simulación, navegación y acciones verificables.

1. 136 — Arquitectura percepción-planificación-acción
2. 137 — Sensores, actuadores y fusión
3. 138 — Localización, mapeo y SLAM
4. 139 — Planificación de movimiento y navegación
5. 140 — Control clásico y control aprendido
6. 141 — Aprendizaje por imitación
7. 142 — Simulación, sim-to-real y digital twins
8. 143 — Robots colaborativos y seguridad física
9. 144 — Computer use basado en visión
10. 145 — Agentes de navegador
11. 146 — Automatización de escritorio y RPA agéntica
12. 147 — Proyecto: agente que actúa con límites

136 — Arquitectura percepción-planificación-acción

Parte: 11 — IA encarnada, robótica y uso de computadores
Nivel: experto · **Horas estimadas:** 6
Laboratorio: `robotics` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **arquitectura percepción-planificación-acción** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar arquitectura percepción-planificación-acción usando los conceptos `perception`, `planning`, `action`, `feedback`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`perception`, `planning`, `action`, `feedback`

🗺️ Ubicación en el mapa de la IA

La robótica obliga a la IA a cerrar el lazo con el mundo físico: ya no basta con producir una respuesta, hay que percibir, decidir y actuar bajo ruido, latencia y consecuencias irreversibles. Esta clase abre la Parte 11 con la pregunta arquitectónica fundamental — ¿cómo se organiza un agente que actúa? — y su respuesta condiciona todo lo que sigue: fusión sensorial (137), SLAM (138), planificación (139) y control (140). El mismo dilema arquitectónico reaparece en los agentes de *computer use* (141-144), donde la "percepción" es un screenshot y la "acción" es un clic.

📖 Fundamentos

🔄 El ciclo percepción-planificación-acción

Un robot (o cualquier agente encarnado) opera en un lazo cerrado con su entorno:

```
mundo --> sensores --> PERCEPCIÓN --> estado estimado
estado --> PLANIFICACIÓN --> plan / decisión
plan --> ACCIÓN (actuadores) --> el mundo cambia --> (feedback) --> sensores ...
```

- **Percepción**: transforma lecturas crudas de sensores (píxeles, distancias, encoders) en una representación del estado del mundo. Siempre es incompleta y ruidosa: el robot nunca conoce el estado real, solo una estimación.
- **Planificación**: decide qué hacer dado el estado estimado y un objetivo. Puede ir desde una regla reactiva (`si obstáculo, gira`) hasta búsqueda en un espacio de estados (A*, clase 139).
- **Acción**: convierte la decisión en comandos de actuadores (motores, pinzas, o un clic de ratón en computer use). La ejecución física introduce su propio error: las ruedas patinan, los motores tienen holgura.
- **Feedback**: el resultado de la acción vuelve por los sensores y corrige el siguiente ciclo. Sin retroalimentación el sistema opera en lazo abierto y el error se acumula sin límite.

🏛️ Sense-Plan-Act: el paradigma deliberativo

La arquitectura clásica (Shakey, SRI, años 70) es una tubería secuencial: percibir todo → construir un modelo del mundo → planificar sobre el modelo → ejecutar. Su fortaleza es el razonamiento global: puede garantizar planes óptimos sobre su modelo. Sus debilidades históricas:

1. **Latencia**: si planificar toma segundos, el mundo cambió cuando el plan llega a los motores.
2. **Fragilidad del modelo**: todo error de percepción se propaga al plan.
3. **Cuello de botella único**: si el planificador falla, el robot se detiene.

🪰 Subsumption: el paradigma reactivo de Brooks

Rodney Brooks (1986) propuso invertir el diseño: en lugar de una tubería vertical, capas horizontales de comportamiento, cada una conectando sensores a actuadores directamente y sin modelo central del mundo ("the world is its own best model"). Las capas superiores *subsumen* (inhiben o modulan) a las inferiores:

```
capa 2: explorar  ----inhibe---+
capa 1: vagar     ----inhibe---+--> actuadores
capa 0: evitar choques --------+
```

Cada capa es una máquina de estados finitos aumentada; el sistema es robusto (si falla "explorar", "evitar choques" sigue funcionando) y de latencia mínima, pero no puede razonar sobre objetivos de largo plazo ni garantizar optimalidad.

🧬 Arquitecturas híbridas de tres capas

La práctica moderna combina ambos extremos en una jerarquía por frecuencia:

- **Capa deliberativa** (~0.1-1 Hz): planificación global, misión, mapas.
- **Capa ejecutiva / secuenciador** (~1-10 Hz): descompone el plan en comportamientos, monitoriza fallos, replanifica.
- **Capa reactiva de control** (~100-1000 Hz): evitación de obstáculos, control de motores, paradas de emergencia.

ROS 2 materializa este patrón: nodos de percepción publican estimaciones, `Nav2` planifica, y controladores de baja latencia cierran el lazo. La regla de diseño clave: **la seguridad vive en la capa rápida**, nunca en la deliberativa.

🧮 Ejemplo trabajado

Robot aspirador en una rejilla 1D de 5 celdas: `[A, B, C, D, E]`. Está en `C`, hay suciedad en `A` y `E`, y un obstáculo móvil aparece en `D` en el paso 3.

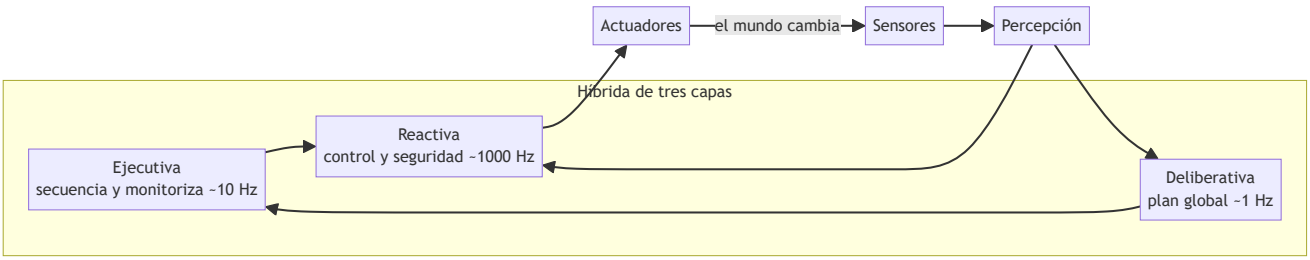
Diseño sense-plan-act puro: percibe todo, planifica la ruta óptima `C→D→E (limpia)→D→C→B→A (limpia)` (6 movimientos, óptimo). En el paso 3 el obstáculo bloquea `D`: el plan es inválido, el robot debe detenerse y replanificar completo (`C→B→A→...`), pagando la latencia de planificación cada vez que el mundo cambia.

Diseño subsumption: capa 0 = "si celda actual sucia, limpia"; capa 1 = "si obstáculo delante, invierte dirección"; capa 2 = "avanza en la dirección actual". Paso a paso: `C→D`, obstáculo → invierte, `D→C→B→A` (limpia), pared → invierte, `B→C→D→E` (limpia). Total 8 movimientos: subóptimo, pero **nunca se detuvo** y no necesitó replanificar.

Híbrido: la capa deliberativa mantiene la ruta óptima; la capa reactiva esquiva el obstáculo localmente y notifica al planificador solo si el desvío supera un umbral. Se obtienen ~6-7 movimientos con robustez reactiva. Este trade-off cuantificado (optimalidad vs. latencia vs. robustez) es el argumento central de la clase.

📊 Propiedades y comparación

Propiedad	Sense-Plan-Act	Subsumption (Brooks)	Híbrida (3 capas)
Modelo del mundo	Central y explícito	Ninguno	Por capa (global arriba, local abajo)
Latencia de reacción	Alta (segundos)	Mínima (ms)	ms en la capa reactiva
Optimalidad del plan	Sí, sobre su modelo	No garantizada	Aproximada
Robustez ante fallos parciales	Baja (cuello de botella)	Alta (capas independientes)	Alta
Objetivos de largo plazo	Naturales	Muy difíciles	Naturales
Ejemplo histórico	Shakey (SRI)	Genghis, Roomba temprano	ROS 2 + Nav2



⚠️ Errores conceptuales frecuentes

1. **"El robot conoce el estado del mundo."** Solo tiene una estimación ruidosa e incompleta; toda la arquitectura existe para gestionar esa incertidumbre.
2. **"Reactivo = primitivo, deliberativo = avanzado."** Son puntos de un trade-off latencia/optimalidad; un Roomba reactivo limpia casas reales que un planificador lento no limpiaría.
3. **"Subsumption no tiene representación."** No tiene *modelo central*; cada capa sí tiene estado interno (máquinas de estados finitas aumentadas).
4. **"Con un buen planificador no hace falta feedback."** En lazo abierto el error de actuación se acumula sin cota; el feedback es lo que hace viable actuar en el mundo físico.
5. **"Esto solo aplica a robots."** Un agente de computer use tiene exactamente el mismo lazo: screenshot (percepción) → decidir (planificación) → clic (acción) → nuevo screenshot (feedback).

🚀 Del aprendizaje a la operación

Entre esta simulación didáctica y un robot real median: drivers y sincronización de sensores reales (timestamps, calibración), un middleware con garantías de tiempo real (ROS 2 con DDS y QoS), watchdogs y paradas de emergencia certificadas en la capa rápida, pruebas HIL (hardware-in-the-loop) y una estrategia de degradación segura cuando la percepción se degrada. Ninguna de esas piezas aparece en el laboratorio y todas son obligatorias en operación.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("robotics")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Brooks, R. (1986). A Robust Layered Control System for a Mobile Robot. IEEE J. Robotics and Automation. DOI 10.1109/JRA.1986.1087032 — uso: fuente primaria del mecanismo estudiado
- Siciliano, B. & Khatib, O. (eds.). Springer Handbook of Robotics, 2e — caps. de arquitecturas robóticas — uso: referencia consultada en su fuente original
- Thrun, S., Burgard, W. & Fox, D. Probabilistic Robotics — cap. 1 (introducción al lazo percepción-acción) — uso: referencia consultada en su fuente original
- Russell, S. & Norvig, P. AIMA 4e — cap. 26, Robotics — uso: desarrollo extendido del tema
- ROS 2 Documentation — uso: referencia consultada en su fuente original
- Nav2 (Navigation2) Documentation — uso: referencia consultada en su fuente original

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P97 · Un sistema de control por capas robusto para un robot móvil	1986	Demuestra que un robot puede comportarse de forma competente sin modelo del mundo, sin planificador y sin representación central.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Sebastian Thrun — *Probabilistic Robotics*	2005	ISBN 9780262201629 · web de la obra	citada en las referencias de esta clase · cap. 1 · obra de referencia de la parte 11
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · cap. 26

📝 Evaluación completa

❓ Preguntas

1. Define arquitectura percepción-planificación-acción sin usar una marca o framework como definición.
2. Explica la relación entre perception, planning, action, feedback.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

137 — Sensores, actuadores y fusión

Parte: 11 — IA encarnada, robótica y uso de computadores
Nivel: experto · **Horas estimadas:** 6
Laboratorio: `robotics` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **sensores, actuadores y fusión** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar sensores, actuadores y fusión usando los conceptos `sensors`, `actuators`, `fusion`, `calibration`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`sensors`, `actuators`, `fusion`, `calibration`

🗺️ Ubicación en el mapa de la IA

Toda la robótica descansa sobre una verdad incómoda: los sensores mienten un poco y los actuadores obedecen a medias. Esta clase introduce el tratamiento probabilístico de esa incertidumbre — calibración y fusión de sensores, con el filtro de Kalman como pieza central — que la clase 136 dio por supuesto al hablar de "percepción". Es el prerequisito directo de SLAM (138), donde la fusión se extiende de un estado escalar a la pose y el mapa completos, y del control (140), que necesita estimaciones limpias para cerrar el lazo.

📖 Fundamentos

📡 Sensores: qué miden y cómo fallan

- **Propioceptivos** (miden el propio robot): encoders de rueda, IMU (acelerómetro + giróscopo), sensores de corriente. Rápidos y siempre disponibles, pero **integran error**: la odometría deriva sin cota.
- **Exteroceptivos** (miden el mundo): LiDAR, cámaras, ultrasonido, GPS. Aportan referencias absolutas o relativas al entorno, pero sufren oclusión, ruido dependiente de la escena y frecuencias bajas.

Todo sensor se caracteriza por: rango, resolución, frecuencia, **sesgo** (error sistemático, se corrige con calibración) y **varianza** (error aleatorio, se gestiona con fusión). Modelo estándar de una medición:

```
z = h(x) + b + v      donde  h(x): función de observación del estado real x
                             b: sesgo (calibrable), v ~ N(0, R): ruido
```

⚙️ Actuadores

Motores DC/BLDC con reductora, servomotores y actuadores lineales convierten comandos en movimiento. Sus imperfecciones (holgura o *backlash*, saturación, zona muerta, retardo) son la razón de que ordenar "avanza 1 m" no produzca exactamente 1 m — y de que el feedback de la clase 136 sea imprescindible.

💉 Calibración

Estimar y corregir el sesgo `b` y los parámetros de `h` con mediciones de referencia (patrón de ajedrez para cámaras, superficie plana para IMU, giro de 360° para odometría). La calibración elimina error **sistemático**; jamás elimina la varianza `R`.

🔀 Fusión de sensores: por qué promediar con pesos

Dos mediciones independientes del mismo escalar, con varianzas σ_1^2 y σ_2^2, se combinan de forma óptima (mínima varianza) ponderando por la inversa de la varianza:

$$\hat{x} = \frac{\sigma_2^2 z_1 + \sigma_1^2 z_2}{\sigma_1^2 + \sigma_2^2} \qquad \sigma^2 = \frac{\sigma_1^2 \sigma_2^2}{\sigma_1^2 + \sigma_2^2} < \min(\sigma_1^2, \sigma_2^2)$$

La fusión **siempre reduce la incertidumbre** por debajo del mejor sensor individual. El filtro de Kalman generaliza esta idea a estados dinámicos.

📉 Filtro de Kalman (versión conceptual 1D)

Mantiene una creencia gaussiana `N(x̂, P)` y alterna dos pasos:

```
PREDICCIÓN (usa el modelo de movimiento, crece la incertidumbre):
    x̂⁻ = a·x̂ + b·u              P⁻ = a²·P + Q

CORRECCIÓN (usa la medición, decrece la incertidumbre):
    K  = P⁻ / (P⁻ + R)           # ganancia de Kalman ∈ (0, 1)
    x̂  = x̂⁻ + K·(z − x̂⁻)        # se mueve hacia z según la confianza
    P  = (1 − K)·P⁻
```

Lectura de la ganancia: si el sensor es preciso (R pequeño), K→1 y la estimación sigue a la medición; si el sensor es ruidoso (R grande), K→0 y domina el modelo. `Q` es el ruido de proceso: cuánta incertidumbre añade cada paso de movimiento.

🎛️ Filtro complementario

Alternativa barata para fusionar dos sensores con errores en frecuencias distintas (típico IMU): pasa-bajos sobre el sensor con deriva lenta y ruido alto de corto plazo (acelerómetro), pasa-altos sobre el que deriva (giróscopo):

```
ángulo = α·(ángulo + gyro·dt) + (1−α)·ángulo_accel        α ≈ 0.98
```

Sin matrices ni covarianzas: es el 90 % de los drones de hobby.

🧮 Ejemplo trabajado

Robot en un pasillo (posición 1D en metros). Estado inicial `x̂=0, P=1`. Modelo: avanza `u=1` m por paso (`a=1, b=1`), ruido de proceso `Q=0.25`. Sensor de distancia con `R=1`. Medición en t=1: `z=1.2`.

Paso 1 — Predicción:

```
x̂⁻ = 1·0 + 1·1 = 1.0
P⁻ = 1·1 + 0.25 = 1.25
```

Paso 1 — Corrección con z=1.2:

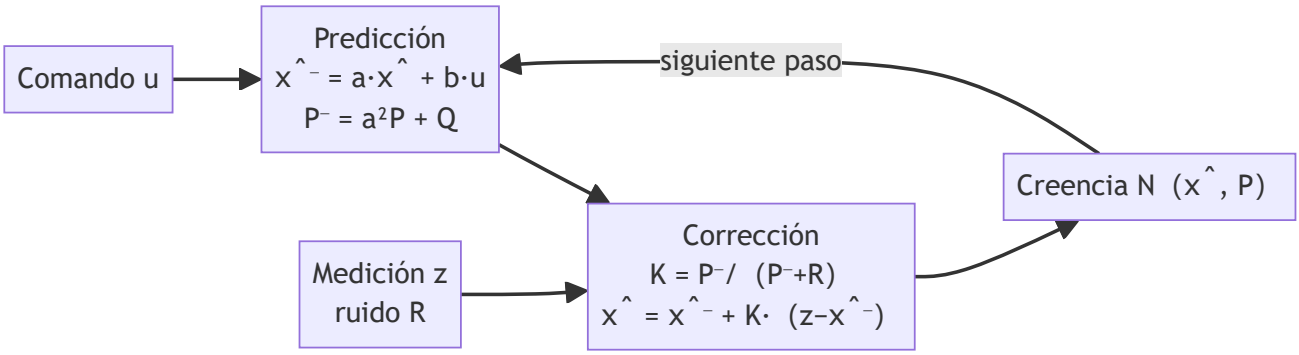
```
K  = 1.25 / (1.25 + 1) = 0.5556
x̂  = 1.0 + 0.5556·(1.2 − 1.0) = 1.111
P  = (1 − 0.5556)·1.25 = 0.5556
```

La estimación (1.111) queda entre el modelo (1.0) y la medición (1.2), más cerca de la medición porque `P⁻ > R`. La incertidumbre cayó de 1.25 a 0.556.

Paso 2 con `z=2.3`: predicción `x̂⁻=2.111, P⁻=0.806`; ganancia `K=0.806/1.806=0.446`; corrección `x̂=2.111+0.446·(2.3−2.111)=2.195`, `P=0.446`. Nota cómo `P` converge hacia un valor estacionario (~0.42 en este sistema): el filtro alcanza un equilibrio entre lo que aporta el modelo y lo que aporta el sensor.

📊 Propiedades y comparación

Método	Estado que maneja	Supuestos	Coste	Cuándo usarlo
Promedio ponderado	Estático	Gaussiano, sin dinámica	O(1)	Fusión puntual de 2+ sensores
Filtro complementario	1 variable con deriva	Errores separables en frecuencia	O(1)	IMU de bajo coste, drones
Kalman (KF)	Vector, dinámica lineal	Lineal + gaussiano	O(n³) por paso	Odometría+GPS, tracking
EKF	No lineal suave	Linealización local válida	O(n³)	SLAM (clase 138), IMU 3D
Filtro de partículas	No lineal, multimodal	Muestreo suficiente	O(n·partículas)	Localización global ambigua



⚠️ Errores conceptuales frecuentes

1. **"Un sensor caro elimina la necesidad de fusión."** Todo sensor tiene varianza y modos de fallo; fusionar dos sensores mediocres e independientes suele rendir más que uno excelente sin redundancia.
2. **"La calibración corrige el ruido."** Corrige el **sesgo** (sistemático); la varianza aleatoria solo se reduce con fusión o promediado temporal.
3. **"El filtro de Kalman predice el futuro."** Estima el estado *presente* combinando modelo y medición; la predicción es solo el paso intermedio.
4. **"K es un parámetro que se sintoniza a mano."** K se calcula en cada paso a partir de P y R; lo que se sintoniza son Q y R.
5. **"Si P converge, la estimación es correcta."** P pequeño significa *consistencia interna*; con un modelo o calibración erróneos el filtro converge con confianza a un valor equivocado (filtro sobreconfiado).

🚀 Del aprendizaje a la operación

En un sistema real faltan: sincronización temporal de sensores (timestamps, interpolación, compensación de latencia), detección y rechazo de *outliers* (un GPS multipath puede corromper el filtro entero), estimación honesta de Q y R a partir de datos, transformaciones entre marcos de referencia (TF en ROS 2) y monitorización de la salud de cada sensor con degradación controlada cuando uno falla. La versión 1D a mano es el modelo mental; producción es gestión de covarianzas en 15+ dimensiones con datos sucios.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("robotics")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;

- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Thrun, S., Burgard, W. & Fox, D. Probabilistic Robotics — caps. 2-3 (filtros bayesianos y de Kalman) — uso: referencia consultada en su fuente original
- Kalman, R. E. (1960). A New Approach to Linear Filtering and Prediction Problems. J. Basic Engineering. DOI 10.1115/1.3662552 — uso: fuente primaria del mecanismo estudiado
- Siciliano, B. & Khatib, O. (eds.). Springer Handbook of Robotics, 2e — parte C, Sensing and Perception — uso: referencia consultada en su fuente original
- KalmanFilter.NET — tutorial ilustrado del filtro de Kalman (Alex Becker) — uso: referencia consultada en su fuente original
- ROS 2 — robot_localization (fusión EKF/UKF en producción) — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P96 · Un nuevo enfoque para los problemas de filtrado y predicción lineales	1960	Fusiona un modelo del movimiento con un sensor ruidoso ponderando cada fuente por su propia incertidumbre, y lo hace de forma recursiva.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Sebastian Thrun — *Probabilistic Robotics*	2005	ISBN 9780262201629 · web de la obra	citada en las referencias de esta clase · caps. 2-3 · obra de referencia de la parte 11

📝 Evaluación completa

❓ Preguntas

1. Define sensores, actuadores y fusión sin usar una marca o framework como definición.
2. Explica la relación entre sensors, actuators, fusion, calibration.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

138 — Localización, mapeo y SLAM

Parte: 11 — IA encarnada, robótica y uso de computadores
Nivel: experto · **Horas estimadas:** 6
Laboratorio: `robotics` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **localización, mapeo y slam** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar localización, mapeo y slam usando los conceptos `localization`, `mapping`, `SLAM`, `uncertainty`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`localization`, `mapping`, `SLAM`, `uncertainty`

🗺️ Ubicación en el mapa de la IA

Con sensores calibrados y fusión (clase 137), el robot puede estimar *cuánto se movió*; pero para navegar necesita saber *dónde está* y *cómo es el mundo*. SLAM (Simultaneous Localization And Mapping) resuelve ambas preguntas a la vez y es uno de los logros centrales de la robótica probabilística: sin él no hay aspiradoras que mapean casas, ni coches autónomos, ni realidad aumentada estable. La pose y el mapa que produce son la entrada directa de la planificación de movimiento (clase 139).

📖 Fundamentos

🧭 Localización, mapeo y el problema del huevo y la gallina

- **Localización**: dado un mapa conocido, estimar la pose `(x, y, θ)` del robot a partir de sensores. Resoluble con filtros (Kalman, partículas — Monte Carlo Localization).
- **Mapeo**: dada una trayectoria conocida, construir el mapa (rejilla de ocupación o conjunto de landmarks).

- **SLAM**: ninguna de las dos cosas se conoce. Para localizarse hace falta el mapa; para mapear hace falta la pose. La solución es estimar **conjuntamente** la distribución `p(pose, mapa | observaciones, controles)`.

📉 Odometría y deriva

La odometría integra el movimiento (encoders, IMU): cada paso añade error, y el error **se acumula sin cota**. Un error angular pequeño es el más dañino: desvía toda la trayectoria posterior de forma proporcional a la distancia recorrida (ver ejemplo trabajado). Por eso la odometría sola nunca basta: hace falta *anclar* la trayectoria a referencias externas (landmarks) y, sobre todo, **cerrar lazos**: reconocer un lugar ya visitado y corregir de golpe toda la deriva acumulada.

📍 EKF-SLAM conceptual

EKF-SLAM extiende el filtro de Kalman (clase 137) a un estado aumentado que contiene la pose del robot y las posiciones de los N landmarks:

```
estado: X = [x, y, θ, l1x, l1y, l2x, l2y, ..., lNx, lNy]   dimensión 3+2N
creencia: N(X̂, P)  con P de tamaño (3+2N)×(3+2N)

por cada paso:
  1. PREDICCIÓN: propaga la pose con el modelo de movimiento; P crece.
  2. ASOCIACIÓN DE DATOS: ¿qué landmark corresponde a cada observación?
  3. CORRECCIÓN: la observación (distancia/ángulo a un landmark) corrige
     la pose Y el landmark; las correlaciones en P propagan la corrección
     al resto del mapa.
```

Dos propiedades clave: (1) la incertidumbre del mapa está **correlacionada** con la de la pose — observar bien un landmark mejora la estimación de otros a través de P; (2) el coste es $O(N^2)$ por paso por el tamaño de P, lo que motivó alternativas (FastSLAM con partículas, graph-SLAM con optimización de grafos, que es lo que usan los sistemas modernos como Cartographer u ORB-SLAM).

🔁 Cierre de lazo (loop closure)

Cuando el robot reconoce un lugar visitado (por apariencia o por geometría), se añade una restricción entre dos poses lejanas en el tiempo. En graph-SLAM esa restricción redistribuye el error por toda la trayectoria: el mapa "encaja" de golpe. Un cierre de lazo **falso** (asociar mal dos lugares parecidos) es catastrófico: corrompe el mapa entero. La asociación de datos es el talón de Aquiles de todo SLAM.

🗺️ Representaciones del mapa

- **Rejilla de ocupación**: cada celda guarda p(ocupada); ideal para LiDAR 2D y planificación. Memoria proporcional al área.
- **Mapa de landmarks**: lista de puntos distintivos; compacto, requiere buenos detectores y asociación.
- **Mapas densos / nubes de puntos / mallas** (SLAM visual 3D): más ricos y más costosos.

🧮 Ejemplo trabajado

Deriva por error angular. Un robot recorre un cuadrado de 10 m de lado usando solo odometría. Su giróscopo tiene un sesgo minúsculo: cada giro de 90° lo ejecuta/mide como 91° (error de 1°).

Tras la primera esquina el rumbo lleva 1° de error; en 10 m de tramo la desviación lateral es aproximadamente:

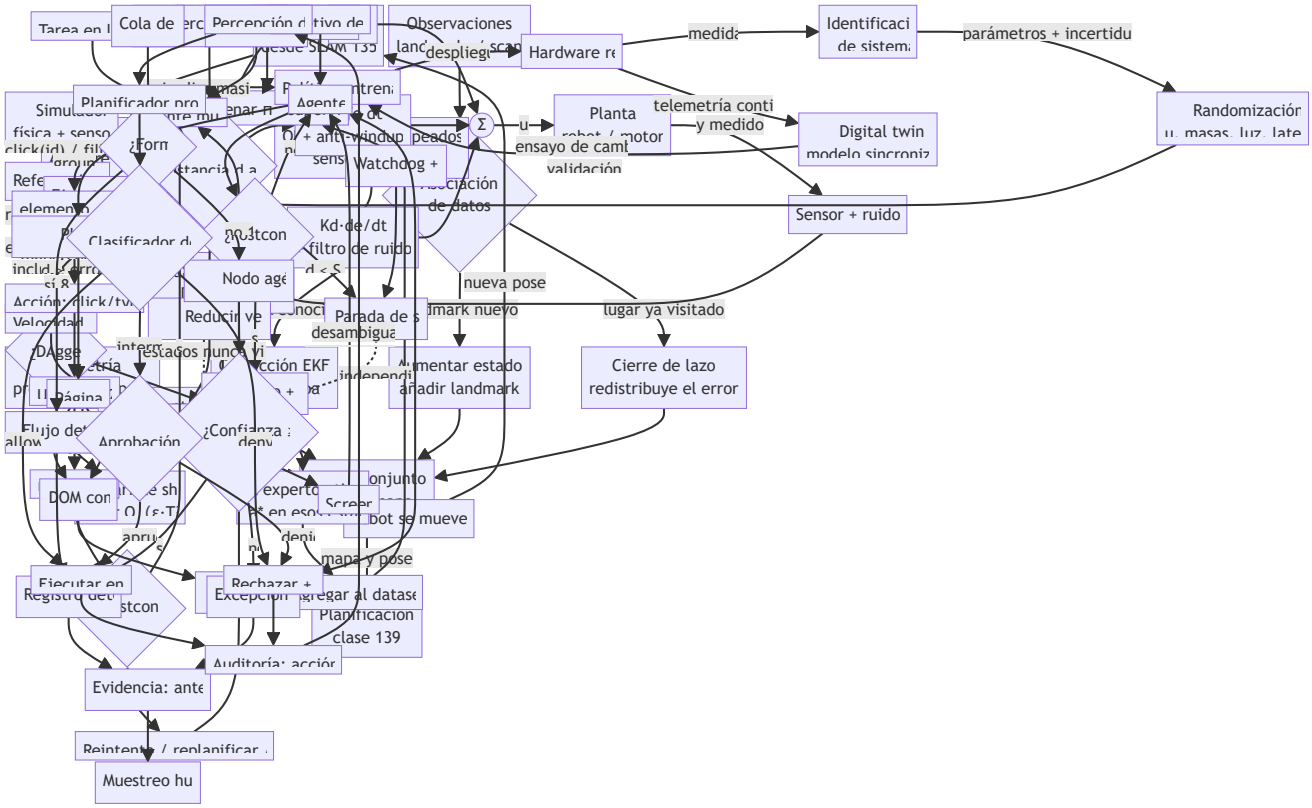
```
desvío ≈ d·sin(ε) = 10·sin(1°) ≈ 0.175 m
```

Los errores angulares se suman esquina tras esquina (1°, 2°, 3°, 4°), y cada tramo hereda el rumbo acumulado. Desvíos laterales por tramo: ~0.175, ~0.349, ~0.523, ~0.698 m. Al "cerrar" el cuadrado, el robot no vuelve al origen: queda a **más de 1 m** del punto de partida (≈1.2 m combinando componentes), con un error de rumbo final de 4°. Con 0.1° de sesgo por giro el error final sería ~0.12 m: la deriva escala linealmente con el sesgo, pero *nunca es cero*.

Si el robot observa un landmark conocido en el origen (una baliza en la esquina de partida), la corrección tipo Kalman con esa observación reduce el error final al nivel del ruido del sensor de la baliza (~centímetros): esto es, en miniatura, un cierre de lazo.

📊 Propiedades y comparación

Enfoque	Estado estimado	Coste por paso	Maneja ambigüedad	Uso típico
Odometría pura	Pose (sin mapa)	O(1)	No; deriva sin cota	Estimación a corto plazo
Localización MCL (partículas)	Pose con mapa dado	O(partículas)	Sí (multimodal)	Robot en mapa conocido
EKF-SLAM	Pose + N landmarks	$O(N^2)$	No (unimodal)	Mapas pequeños de landmarks
FastSLAM	Partículas × mapas	O(P·log N)	Sí	Landmarks, mapas medianos
Graph-SLAM (moderno)	Grafo de poses	Optimización dispersa	Con robustez extra	Cartographer, ORB-SLAM



⚠️ Errores conceptuales frecuentes

1. **"Con buenos encoders no hay deriva."** La deriva es estructural a la integración de ruido: mejores sensores la reducen, jamás la eliminan.
2. **"SLAM = construir un mapa."** Sin la estimación conjunta de la pose no es SLAM; el acoplamiento pose-mapa es precisamente la dificultad.
3. **"El GPS resuelve SLAM."** En interiores no hay GPS; en exteriores su error (metros, multipath) y su frecuencia no bastan para mapear con precisión — se usa como un sensor más dentro de la fusión.
4. **"Más landmarks siempre ayudan."** En EKF-SLAM el coste crece $O(N^2)$ y cada landmark añade riesgo de asociación errónea; la calidad y distintividad importan más que la cantidad.
5. **"Un cierre de lazo siempre mejora el mapa."** Uno falso lo corrompe por completo; los sistemas reales verifican geométricamente cada cierre antes de aceptarlo.

🚀 Del aprendizaje a la operación

Un SLAM operativo exige: asociación de datos robusta (descriptores visuales, verificación geométrica), gestión de mapas de larga duración (el mundo cambia: muebles, iluminación, estaciones), relocalizacón tras secuestro o reinicio, presupuesto de cómputo en el robot (graph-SLAM disperso, no EKF denso) y métricas de calidad del mapa (ATE/RPE contra ground truth). Herramientas como Cartographer, RTAB-Map u ORB-SLAM3 encapsulan una década de ingeniería que el modelo conceptual de esta clase solo esboza.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("robotics")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Thrun, S., Burgard, W. & Fox, D. Probabilistic Robotics — caps. 7-10 (localización y SLAM) — uso: referencia consultada en su fuente original
- Durrant-Whyte, H. & Bailey, T. (2006). Simultaneous Localization and Mapping: Part I. IEEE Robotics & Automation Magazine. DOI 10.1109/MRA.2006.1638022 — uso: fuente primaria del mecanismo estudiado
- Cadena, C. et al. (2016). Past, Present, and Future of SLAM. IEEE Trans. on Robotics. arXiv:1606.05830 — uso: fuente primaria del mecanismo estudiado
- Siciliano, B. & Khatib, O. (eds.). Springer Handbook of Robotics, 2e — cap. de SLAM — uso: referencia consultada en su fuente original
- Nav2 — documentación de localización y mapas — uso: referencia consultada en su fuente original
- Cartographer (Google) — documentación oficial — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P99 · Localización y mapeo simultáneos: parte I	2006	Formaliza el problema circular de la robótica móvil: no se puede localizar sin mapa ni mapear sin localización, y hay que resolver ambos a la vez.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Sebastian Thrun — *Probabilistic Robotics*	2005	ISBN 9780262201629 · web de la obra	citada en las referencias de esta clase · caps. 7-10 · obra de referencia de la parte 11

📝 Evaluación completa

❓ Preguntas

1. Define localización, mapeo y slam sin usar una marca o framework como definición.
2. Explica la relación entre localization, mapping, SLAM, uncertainty.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

139 — Planificación de movimiento y navegación

Parte: 11 — IA encarnada, robótica y uso de computadores
Nivel: experto · **Horas estimadas:** 6
Laboratorio: `search` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **planificación de movimiento y navegación** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar planificación de movimiento y navegación usando los conceptos `path planning`, `navigation`, `obstacles`, `A*`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`path planning`, `navigation`, `obstacles`, `A*`

🗺️ Ubicación en el mapa de la IA

Con una pose y un mapa (clase 138), la pregunta siguiente es *cómo llegar de A a B sin chocar*. La planificación de movimiento conecta la búsqueda clásica en grafos (A*, herencia directa de la IA simbólica) con la geometría del mundo físico a través del espacio de configuración, y añade la capa reactiva local (DWA) que ejecuta el plan entre obstáculos móviles. Sus salidas alimentan el control de bajo nivel (clase 140), y sus ideas reaparecen en los agentes digitales: planificar una secuencia de clics es también búsqueda en un espacio de estados.

📖 Fundamentos

📐 Espacio de configuración (C-space)

Una configuración `q` es el vector mínimo que fija la postura del robot (para un robot móvil plano: `(x, y, θ)`; para un brazo de 6 ejes: 6 ángulos). El **C-space** es el conjunto de todas las configuraciones; se divide en `C_libre` (sin colisión) y `C_obs`. El truco fundamental: planificar en C-space convierte a un robot con forma y

tamaño en **un punto**, a cambio de *inflar* los obstáculos con la geometría del robot (suma de Minkowski). Un pasillo de 80 cm con un robot de 60 cm de diámetro se convierte en un pasillo libre de 20 cm para un punto.

⭐ Búsqueda en rejilla: A*

Discretiza `C_libre` en celdas y busca el camino de coste mínimo. A* expande nodos por prioridad `f(n) = g(n) + h(n)`:

```
g(n): coste real acumulado desde el inicio hasta n
h(n): heurística — estimación del coste restante hasta la meta
      admisible si NUNCA sobreestima (p. ej. distancia euclídea)
```

Con heurística admisible (y consistente), A *es completo y óptimo en la rejilla. Con* `h=0` *degenera en Dijkstra (más expansiones); con* `h` *sobreestimada (A* ponderado) encuentra caminos más rápido pero pierde la garantía de optimalidad. El coste práctico explota con la dimensión: una rejilla de 100 celdas por eje tiene 10^4 nodos en 2D, 10^{12} en 6D — la **maldición de la dimensionalidad**.

🎄 Métodos basados en muestreo: RRT y PRM

Para C-spaces de alta dimensión (brazos, drones) no se discretiza: se muestrea.

```
RRT (Rapidly-exploring Random Tree):
  1. árbol T inicializado en q_inicio
  2. repetir: muestrea q_rand en C-space
     q_near  <- nodo de T más cercano a q_rand
     q_new   <- avanza desde q_near hacia q_rand un paso δ
     si el segmento q_near→q_new está libre de colisión: añade q_new a T
  3. parar cuando T alcanza la región de la meta
```

RRT es **probabilísticamente completo** (si existe solución, la probabilidad de encontrarla tiende a 1 con las muestras) pero **no óptimo**: produce caminos dentados que requieren suavizado. RRT* añade recableado local y converge a la solución óptima. PRM construye un grafo de muestras reutilizable para múltiples consultas en el mismo mapa.

🚙 Navegación local: DWA (Dynamic Window Approach)

El plan global no basta: el mundo tiene obstáculos que el mapa no conocía. DWA decide, a cada ciclo de control (~10-20 Hz), el par velocidad lineal/angular `(v, ω)`:

1. **Ventana dinámica**: solo considera velocidades alcanzables en el próximo instante dado el estado actual y los límites de aceleración del robot.
2. Simula la trayectoria corta (1-3 s) de cada par `(v, ω)` candidato.
3. Puntúa cada trayectoria: avance hacia la meta + distancia a obstáculos + velocidad, y descarta las que colisionan.
4. Ejecuta el mejor par y repite.

La arquitectura resultante es el patrón estándar (Nav2): **planificador global (A*) a ~1 Hz + planificador local (DWA/TEB/MPPI) a ~20 Hz**, exactamente la jerarquía deliberativo/reactivo de la clase 136.

🧮 Ejemplo trabajado

Rejilla 5×5, inicio `S=(0,0)`, meta `G=(4,4)`, movimiento en 4 direcciones (coste 1), obstáculos en `(1,1), (2,1), (3,1), (1,3), (2,3)`. Heurística: distancia Manhattan `h = |4−x| + |4−y|` (admisible con 4 vecinos).

```
y=4  .  .  .  .  G        S=(0,0) abajo-izquierda
y=3  .  #  #  .  .        # = obstáculo
y=2  .  .  .  .  .
y=1  .  #  #  #  .
y=0  S  .  .  .  .
```

Expansión de A *(nodos con `f = g + h`): `S` tiene `f = 0 + 8 = 8`. Sus vecinos `(1,0)` y `(0,1)` tienen `f = 1 + 7 = 8`. A* sigue expandiendo la franja `f=8`: todo camino sin retrocesos mide exactamente 8 pasos… pero los obstáculos obligan a comprobar cuál sobrevive. El muro `y=1` deja un único hueco en `x=0` y `x=4`; el muro `y=3` deja huecos en `x=0`, `x=3` y `x=4`. Un camino de coste 8: `(0,0)→(0,1)→(0,2)→(1,2)→(2,2)→(3,2)→(3,3)→(4,3)→(4,4)` (sube por la izquierda, cruza el pasillo central `y=2` y sale por el hueco `x=3`). A* lo encuentra expandiendo ~12-15 nodos de los 20 libres, sin visitar las esquinas irrelevantes; Dijkstra (`h=0`) habría expandido prácticamente todos. Verificación de optimalidad: la Manhattan pura es 8 y el camino mide 8 ⇒ ningún desvío fue necesario y el resultado es óptimo.

📊 Propiedades y comparación

Método	Completitud	Optimalidad	Coste	Dimensión práctica	Rol
Dijkstra	Completo (rejilla)	Óptimo	Alto (sin guía)	2-3D	Global, base
A*	Completo (rejilla)	Óptimo con h admisible	Medio	2-4D	Global estándar
RRT	Probabilística	No	Bajo por muestra	6D+	Global en alta dimensión
RRT*	Probabilística	Asintótica	Medio	6D+	Global con calidad
PRM	Probabilística	Con densidad	Precómputo alto	6D+	Multi-consulta
DWA	Local (no global)	No	Muy bajo por ciclo	2D + dinámica	Local reactivo

⚠️ Errores conceptuales frecuentes

1. **"A* planifica para el robot con su forma real."** A* planifica para un punto; la forma se maneja antes, inflando obstáculos en el C-space. Olvidar la inflación produce planes que rozan paredes.
2. **"RRT encuentra el camino óptimo."** RRT solo garantiza *encontrar* un camino (probabilísticamente); la optimalidad exige RRT* o post-procesado.
3. **"Una heurística mayor siempre acelera."** Solo mientras sea admisible; sobreestimar acelera pero puede devolver caminos subóptimos sin aviso.
4. **"Con un buen plan global sobra el planificador local."** El mapa siempre está incompleto o desactualizado; sin capa local el primer peatón invalida todo.
5. **"Camino geométrico = trayectoria ejecutable."** El camino ignora dinámica (velocidades, aceleraciones, radio de giro); convertirlo en trayectoria temporizada y factible es un paso adicional que DWA resuelve localmente.

🚀 Del aprendizaje a la operación

En producción se añaden: costmaps por capas con inflación y decaimiento temporal (Nav2), replanificación continua con presupuesto de tiempo, restricciones cinodinámicas reales (masa, fricción, límites de par), recuperación ante bloqueos (comportamientos de escape), y validación estadística de tasas de éxito/colisión en flotas simuladas antes de tocar un robot físico. El salto de "A* en rejilla 5×5" a "robot en un almacén con personas" está en esas capas, no en el algoritmo de búsqueda.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("search")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- LaValle, S. M. Planning Algorithms (Cambridge UP, 2006) — libro completo gratuito — uso: referencia consultada en su fuente original
- Hart, P., Nilsson, N. & Raphael, B. (1968). A Formal Basis for the Heuristic Determination of Minimum Cost Paths. DOI 10.1109/TSSC.1968.300136 — uso: fuente primaria del mecanismo estudiado
- LaValle, S. M. (1998). Rapidly-Exploring Random Trees: A New Tool for Path Planning (informe técnico, Iowa State) — uso: referencia consultada en su fuente original
- Fox, D., Burgard, W. & Thrun, S. (1997). The Dynamic Window Approach to Collision Avoidance. IEEE RAM. DOI 10.1109/100.580977 — uso: fuente primaria del mecanismo estudiado
- Russell, S. & Norvig, P. AIMA 4e — cap. 3 (búsqueda) y cap. 26 (robótica) — uso: desarrollo extendido del tema

- Nav2 — Navigation Concepts (planificador global y local) — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P98 · RRT-Connect: un enfoque eficiente para planificación de caminos de consulta única	2000	Planifica en espacios continuos de muchas dimensiones sin discretizarlos, creciendo un árbol hacia muestras aleatorias.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · cap. 3
Sebastian Thrun — *Probabilistic Robotics*	2005	ISBN 9780262201629 · web de la obra	obra de referencia de la parte 11 · toda la parte

📝 Evaluación completa

❓ Preguntas

1. Define planificación de movimiento y navegación sin usar una marca o framework como definición.
2. Explica la relación entre path planning, navigation, obstacles, A*.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

140 — Control clásico y control aprendido

Parte: 11 — IA encarnada, robótica y uso de computadores
Nivel: experto · **Horas estimadas:** 6
Laboratorio: `robotics` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **control clásico y control aprendido** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar control clásico y control aprendido usando los conceptos `PID`, `control`, `policy`, `stability`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`PID`, `control`, `policy`, `stability`

🗺️ Ubicación en el mapa de la IA

La planificación (clase 139) produce trayectorias; el **control** es quien las ejecuta cientos de veces por segundo contra la física real. El PID — formulado hace un siglo — sigue gobernando la inmensa mayoría de lazos industriales, y entender por qué funciona (y cuándo no) es requisito para apreciar qué aporta el control aprendido por refuerzo, que hoy domina en locomoción de cuadrúpedos y manipulación diestra. Esta clase une la robótica con la Parte de RL del programa: una política aprendida *es* un controlador.

📖 Fundamentos

🎛️ El lazo de control y el error

Un controlador recibe una referencia `r(t)` (consigna) y el estado medido `y(t)`, calcula el error `e(t) = r(t) - y(t)` y produce una acción `u(t)` que empuja el sistema hacia la referencia. Métricas del lazo: tiempo de subida, sobreimpulso (*overshoot*), tiempo de asentamiento y error estacionario.

🧮 PID: proporcional, integral, derivativo

```
u(t) = Kp·e(t) + Ki·∫e(τ)dτ + Kd·de/dt

forma discreta (periodo Δt):
u_k = Kp·e_k + Ki·Σ(e_i·Δt) + Kd·(e_k − e_{k−1})/Δt
```

- **P**: reacciona al error presente. Solo-P deja **error estacionario** ante perturbaciones constantes (necesita error para generar acción) y demasiado Kp produce oscilación.
- **I**: acumula el pasado y elimina el error estacionario; en exceso causa sobreimpulso y *windup* (la integral se satura durante transitorios largos y luego descarga de golpe — se mitiga con anti-windup: limitar la integral).
- **D**: anticipa el futuro con la pendiente del error; amortigua oscilaciones, pero **amplifica el ruido** de medición (en la práctica se filtra o se deriva la medición, no el error).

Sintonización: reglas empíricas (Ziegler-Nichols: subir Kp hasta oscilación sostenida en Ku con periodo Tu; usar `Kp=0.6Ku, Ki=1.2Ku/Tu, Kd=0.075Ku·Tu`), o iterar P→PD→PID validando cada métrica. El ejemplo trabajado muestra la iteración manual completa.

⚖️ Límites del control clásico

PID es lineal, monovariable (SISO) y sin modelo: excelente para lazos individuales (temperatura, velocidad de una rueda), insuficiente cuando hay fuertes acoplamientos no lineales (un cuadrúpedo con 12 articulaciones y contactos intermitentes), restricciones activas o dinámica difícil de modelar. El escalón intermedio es el control con modelo (LQR: óptimo para dinámica lineal y coste cuadrático; MPC: optimiza en horizonte con restricciones).

🤖 Control aprendido (RL)

Una política `π(a|s)` entrenada con refuerzo (PPO, SAC) reemplaza al controlador: el "diseño" se convierte en la definición de la función de recompensa y del entorno de entrenamiento (simulación masiva, luego sim-to-real — clase 142). Ventajas: maneja no linealidades, contactos y alta dimensión; descubre estrategias no obvias. Costes: sin garantías formales de estabilidad, hambre de datos, riesgo de *reward hacking*, y comportamiento pobre fuera de la distribución de entrenamiento. El patrón operativo actual es **híbrido**: la política RL genera consignas de alto nivel y PIDs de bajo nivel las ejecutan en cada articulación, con envolventes de seguridad clásicas alrededor (límites de par, watchdogs).

🧮 Ejemplo trabajado

Control de velocidad de crucero simplificado. Dinámica discreta (`Δt = 0.1 s`): `v_{k+1} = v_k + 0.1·(u_k − 0.5·v_k)` (el término `−0.5·v` es la fricción). Referencia `r = 10 m/s`, `v_0 = 0`.

Intento 1 — solo P, `Kp = 1`: en equilibrio `u = 0.5·v` (la acción debe compensar la fricción). Con `u = 1·e`: `e = 0.5·v ⇒ 10 − v = 0.5·v ⇒ v_∞ = 6.67 m/s`. **Error estacionario de 3.33 m/s**: el P solo llega donde el error genera exactamente la acción que pide la fricción.

Intento 2 — `Kp = 5`: `10 − v = 0.1·v ⇒ v_∞ = 9.09`. Menos error (0.91) pero respuesta más brusca; con Kp muy grande y Δt finito el lazo discreto llega a oscilar. Subir la ganancia *reduce* el error estacionario pero *nunca* lo elimina.

Intento 3 — PI, `Kp = 2, Ki = 1`: la integral acumula el error y aporta en equilibrio exactamente `u = 0.5·10 = 5` con `e = 0`. El error estacionario desaparece; aparece un ligero sobreimpulso (~5-10 %) mientras la integral se descarga. Si se observa windup al arrancar desde v=0 (integral crece durante todo el transitorio), se limita la integral o se congela mientras `|e|` es grande.

Intento 4 — PID, añadir `Kd = 0.5`: el término derivativo frena la aproximación y recorta el sobreimpulso a ~1-2 % con tiempo de asentamiento similar. Este es el flujo de sintonización manual estándar: P para velocidad de respuesta, I para el error final, D para amortiguar.

📊 Propiedades y comparación

Enfoque	Modelo requerido	Garantías	No linealidad / contactos	Coste de diseño	Uso típico
PID	Ninguno	Empíricas (por lazo)	Pobre	Horas	90 % de lazos industriales
LQR	Lineal exacto	Óptimo (coste cuadrático)	No	Días (modelado)	Estabilización local
MPC	Modelo + restricciones	Restricciones garantizadas	Media	Semanas	Automoción, química
Política RL	Simulador	Ninguna formal	Excelente	Semanas + GPU	Locomoción, manipulación
Híbrido RL+PID	Simulador + lazos	Envolvente clásica	Excelente	Semanas	Cuadrúpedos comerciales

⚠️ Errores conceptuales frecuentes

1. **"Subiendo Kp lo suficiente desaparece el error estacionario."** Solo lo reduce asintóticamente; eliminarlo ante perturbación constante requiere el término integral (o feedforward).
2. **"El término D predice el futuro del sistema."** Solo extrapola la pendiente del error; con medición ruidosa esa pendiente es basura amplificada — de ahí que muchos lazos industriales sean solo PI.
3. **"Una política RL entrenada es estable."** No hay garantía formal; la estabilidad empírica en simulación no se transfiere automáticamente al hardware ni a estados fuera de distribución.
4. **"PID está obsoleto frente a RL."** Los robots comerciales con RL lo usan *encima* de PIDs articulares; el clásico ejecuta, el aprendido decide.
5. **"Sintonizar = probar valores al azar."** Existe una lógica causal: P marca la agresividad, I el régimen final, D el amortiguamiento; cada síntoma (oscilación, error residual, sobreimpulso) señala el

término a tocar.

🚀 Del aprendizaje a la operación

Entre este modelo didáctico y un controlador desplegado median: identificación del sistema real (retardos, saturaciones, zonas muertas que el modelo lineal ignora), ejecución con periodo garantizado (RT, prioridades), anti-windup y *bumpless transfer* entre modos manual/automático, límites de seguridad independientes del controlador (clase 143) y, para políticas RL, validación sim-to-real con randomización de dominio (clase 142) más un plan de reversión a control clásico cuando la política sale de su envolvente validada.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("robotics")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Åström, K. J. & Murray, R. M. Feedback Systems: An Introduction for Scientists and Engineers — PDF oficial gratuito (Caltech) — uso: referencia consultada en su fuente original
- Gymnasium — entornos de control clásico (CartPole, Pendulum) para practicar control aprendido — uso: referencia consultada en su fuente original
- Sutton, R. & Barto, A. Reinforcement Learning: An Introduction, 2e — caps. 13 y 16 (políticas y aplicaciones de control) — uso: desarrollo extendido del tema
- Hwangbo, J. et al. (2019). Learning agile and dynamic motor skills for legged robots. Science Robotics. arXiv:1901.08652 — uso: fuente primaria del mecanismo estudiado
- Siciliano, B. & Khatib, O. (eds.). Springer Handbook of Robotics, 2e — parte B, Design y Control — uso: referencia consultada en su fuente original
- ros2_control — documentación oficial — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P102 · Algoritmos de optimización proximal de políticas	2017	Consigue la estabilidad de TRPO con una función objetivo que se implementa en unas líneas y se optimiza con descenso de gradiente corriente.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Sutton, Richard S. y Barto, Andrew G. — *Reinforcement Learning: An Introduction*	2.ª · 2018	ISBN 9780262039246 · web de la obra	citada en las referencias de esta clase · caps. 13 y 16
Sebastian Thrun — *Probabilistic Robotics*	2005	ISBN 9780262201629 · web de la obra	obra de referencia de la parte 11 · toda la parte

📝 Evaluación completa

❓ Preguntas

1. Define control clásico y control aprendido sin usar una marca o framework como definición.
2. Explica la relación entre PID, control, policy, stability.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

141 — Aprendizaje por imitación

Parte: 11 — IA encarnada, robótica y uso de computadores
Nivel: experto · **Horas estimadas:** 6
Laboratorio: `robotics` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **aprendizaje por imitación** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar aprendizaje por imitación usando los conceptos `imitation`, `demonstrations`, `behavior cloning`, `DAgger`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`imitation`, `demonstrations`, `behavior cloning`, `DAgger`

🗺️ Ubicación en el mapa de la IA

Diseñar recompensas para RL (clase 140) es difícil y peligroso; a menudo es más fácil *mostrar* la tarea que especificarla. El aprendizaje por imitación convierte demostraciones humanas en políticas, y hoy es el motor de los avances más visibles de la robótica de manipulación (ALOHA, Diffusion Policy, modelos visión-lenguaje-acción como RT-2 u OpenVLA) y de la conducción autónoma de extremo a extremo. Su patología central — el *covariate shift* — y su cura — DAgger — son también el marco correcto para pensar por qué los agentes LLM entrenados con trazas de demostración fallan al salirse del camino conocido.

📖 Fundamentos

🎭 Formulación

Dado un conjunto de demostraciones `D = {(s_i, a_i)}` generadas por una política experta `π*`, aprender `πˆ` que imite al experto. A diferencia de RL, no hay función de recompensa: la señal es "haz lo que hizo el experto".

📋 Behavioral Cloning (BC)

La aproximación directa: tratar la imitación como **aprendizaje supervisado**.

```
π̂ = argmin_π  Σ_(s,a)∈D  L(π(s), a)     # regresión/clasificación estándar
```

Simple, estable y sorprendentemente eficaz con datos abundantes y buenos. Su defecto estructural no es el modelo sino la **distribución de datos**.

🌊 Covariate shift y error compuesto

El experto solo visita estados "buenos": D cubre la trayectoria correcta y casi nada alrededor. En ejecución, el más mínimo error lleva a `π̂`a un estado ligeramente fuera de la distribución de entrenamiento; allí su error es mayor, lo que la aleja aún más — un bucle de retroalimentación de errores. Ross y Bagnell formalizaron el resultado: si el error de imitación por paso es ε, el coste acumulado en un horizonte T crece como **O(ε·T²)** para BC, frente al O(ε·T) que se obtendría si los errores no se compusieran. La intuición del coche: el experto nunca demostró "cómo volver al carril desde el arcén", porque nunca pisó el arcén.

🔁 DAgger (Dataset Aggregation)

DAgger (Ross, Gordon & Bagnell, 2011 — arXiv:1011.0686) ataca la causa: recolectar datos **en la distribución de la política aprendida**.

```
D <- demostraciones iniciales;  π̂1 <- BC(D)
para i = 1..N:
    ejecuta π̂i en el entorno y registra los estados visitados s
    pregunta al experto la acción correcta a* = π*(s) en ESOS estados
    D <- D ∪ {(s, a*)}
    π̂{i+1} <- entrenar con D completo
```

Al etiquetar los estados que la política realmente visita — incluidos sus errores — el experto enseña a *recuperarse*. Garantía: coste O(ε·T) (lineal, no cuadrático). Coste práctico: exige un experto **interactivo** disponible durante el entrenamiento, lo que es caro con humanos (variantes: expertos algorítmicos en simulación, etiquetado offline, HG-DAgger con intervención humana solo cuando hace falta).

🧰 El panorama moderno

- **Teleoperación de bajo coste** (ALOHA/ALOHA 2) hizo barato recolectar demostraciones bimanuales.
- **Políticas generativas** (Diffusion Policy, ACT) modelan distribuciones multimodales de acción — crucial cuando hay varias maneras válidas de hacer la tarea y promediar entre ellas produce una acción inválida.
- **Inverse RL** infiere la recompensa que explica al experto y luego optimiza con RL: más costoso, pero generaliza mejor la *intención*.

🧮 Ejemplo trabajado

Corredor discreto de 10 celdas (0-9); el experto camina siempre por la celda "centrada" ideal y D contiene solo pares de las celdas de la ruta experta. Una política BC imperfecta acierta la acción del experto con probabilidad 0.95 **dentro** de la distribución, pero en celdas nunca vistas actúa al azar (0.5 de acierto).

Horizonte T=20 pasos. Probabilidad de completar sin salirse jamás: `0.95²⁰ ≈ 0.36`. En el 64 % de los episodios la política pisa al menos una vez una celda fuera de ruta — y ahí su acierto cae a 0.5, con lo que lo más probable es alejarse todavía más: el error se compone. Resultado típico observado: deriva sostenida tras el primer fallo.

Con una ronda de DAgger: la política se ejecuta, visita las celdas de error (las adyacentes a la ruta), el experto etiqueta "vuelve al centro" en esas celdas y se reentrena. Ahora las celdas vecinas están en distribución con acierto 0.95, y un desvío de un paso se corrige con probabilidad alta en vez de amplificarse: la tasa de episodios completados sube drásticamente (en el notebook se simula: de ~35 % a >85 %). La lección cuantitativa: BC falla como T^2 y DAgger lo devuelve a lineal.

📊 Propiedades y comparación

Método	Señal	Experto interactivo	Error en horizonte T	Coste de datos	Cuándo
Behavioral Cloning	(s, a) del experto	No	$O(\varepsilon \cdot T^2)$	Bajo	Datos masivos, tarea corta
DAgger	Etiquetas en estados propios	Sí	$O(\varepsilon \cdot T)$	Medio-alto	Simulador o experto barato
HG-DAgger / intervención	Correcciones puntuales	Parcial	$\sim O(\varepsilon \cdot T)$	Medio	Teleoperación real
Inverse RL	Recompensa inferida	No	Depende de RL	Alto (cómputo)	Generalizar intención
RL puro (ref.)	Recompensa diseñada	No	—	Muy alto (exploración)	Sin experto disponible

⚠️ Errores conceptuales frecuentes

1. **"BC falla porque el modelo es pequeño."** Falla por la distribución de datos: un modelo perfecto en D sigue sin saber qué hacer fuera de D. Más capacidad no cura el covariate shift.
2. **"99 % de accuracy en validación ⇒ la política funciona."** La validación se mide en la distribución del experto; la ejecución ocurre en la de la política. Son distribuciones distintas por definición del problema.
3. **"DAgger necesita mejores demostraciones."** No: necesita etiquetas en *peores* estados — precisamente los que el experto nunca visitaría solo.

4. **"Más demostraciones del experto resuelven lo mismo que DAgger."** Añadir datos de la misma distribución no cubre los estados de error; reduce ε pero mantiene el T^2.
5. **"Imitación = copiar trayectorias."** Se aprende una política estado→acción (con recuperación incluida si los datos la contienen), no una repetición ciega de la secuencia demostrada.

🚀 Del aprendizaje a la operación

Para desplegar imitación real hacen falta: pipelines de teleoperación y curación de demostraciones (calidad y cobertura importan más que cantidad), métricas de éxito en tarea real y no solo pérdida de validación, detección de salida de distribución en línea (para ceder el control antes del desastre), reentrenos periódicos con las intervenciones humanas acumuladas (el patrón HG-DAgger operativo) y evaluación con protocolos ciegos, porque la varianza entre episodios de manipulación es enorme y invita al autoengaño.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("robotics")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Ross, S., Gordon, G. & Bagnell, J. A. (2011). A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning (DAgger). AISTATS. arXiv:1011.0686 — uso: fuente primaria del mecanismo estudiado
- Pomerleau, D. (1988). ALVINN: An Autonomous Land Vehicle in a Neural Network. NeurIPS — el BC seminal de conducción — uso: referencia consultada en su fuente original
- Chi, C. et al. (2023). Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. arXiv:2303.04137 — uso: fuente primaria del mecanismo estudiado
- Zhao, T. et al. (2023). Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware (ALOHA/ACT). arXiv:2304.13705 — uso: fuente primaria del mecanismo estudiado
- Sutton, R. & Barto, A. Reinforcement Learning: An Introduction, 2e — para contrastar con la señal de recompensa — uso: desarrollo extendido del tema

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P101 · Una reducción del aprendizaje por imitación al aprendizaje en línea sin arrepentimiento	2011	Explica por qué la clonación de comportamiento se degrada con el horizonte, y da un algoritmo que reduce el error de orden T^2 a orden T.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Sutton, Richard S. y Barto, Andrew G. — *Reinforcement Learning: An Introduction*	2.ª · 2018	ISBN 9780262039246 · web de la obra	citada en las referencias de esta clase
Sebastian Thrun — *Probabilistic Robotics*	2005	ISBN 9780262201629 · web de la obra	obra de referencia de la parte 11 · toda la parte

📝 Evaluación completa

❓ Preguntas

1. Define aprendizaje por imitación sin usar una marca o framework como definición.
2. Explica la relación entre imitation, demonstrations, behavior cloning, DAgger.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

142 — Simulación, sim-to-real y digital twins

Parte: 11 — IA encarnada, robótica y uso de computadores
Nivel: experto · **Horas estimadas:** 6
Laboratorio: `robotics` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **simulación, sim-to-real y digital twins** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar simulación, sim-to-real y digital twins usando los conceptos `simulation`, `sim-to-real`, `digital twin`, `domain randomization`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`simulation`, `sim-to-real`, `digital twin`, `domain randomization`

🗺️ Ubicación en el mapa de la IA

Las políticas aprendidas de las clases 140-141 necesitan millones de episodios de práctica; el hardware real es lento, caro y se rompe. La simulación resolvió el cuello de botella de datos de la robótica moderna — casi toda política de locomoción o manipulación actual nace en un simulador — pero abrió otro problema: el **reality gap**. Esta clase cubre las técnicas que cruzan ese hueco (domain randomization, identificación de sistema) y el concepto industrial de **digital twin**, y es el puente directo hacia la evaluación segura de agentes que actúan (clases 140 y 144: probar límites en un entorno sintético antes de tocar el mundo).

📖 Fundamentos

🖥️ Por qué simular

Un simulador físico (MuJoCo, Isaac Lab/Gym, PyBullet, Gazebo) integra la dinámica `s_{t+1} = f(s_t, a_t)` con modelos de contacto, fricción y sensores. Aporta: (1) **escala** — miles de entornos en paralelo en GPU, años de experiencia por día; (2) **seguridad** — los fallos no cuestan hardware ni hieren a nadie; (3)

ground truth — la pose exacta está disponible para recompensas y métricas, sin SLAM ni ruido; (4) **reproducibilidad** — misma semilla, mismo episodio (el mismo principio de los labs de este programa).

🕳️ El reality gap

La política sobreajusta a los detalles del simulador: coeficientes de fricción exactos, latencia cero, dinámica de motores idealizada, texturas e iluminación sintéticas. En el robot real todo eso difiere y el rendimiento se derrumba. El gap tiene dos componentes: **dinámico** (física mal modelada: fricción, holguras, latencias) y **perceptivo** (las imágenes reales no se parecen a las renderizadas).

🎲 Domain randomization

Idea (Tobin et al., 2017 — arXiv:1703.06907): en lugar de hacer el simulador perfecto, hacerlo **variado**. Se aleatorizan parámetros en cada episodio:

```
perceptivo: texturas, colores, iluminación, posición de cámara, ruido de imagen
dinámico:   masas ±20 %, fricción, ganancias de motor, latencias, perturbaciones

hipótesis: si la política funciona bajo TODAS las variaciones,
la realidad es "una randomización más" dentro de la distribución vista.
```

Trade-off central: más randomización ⇒ más robustez pero política más conservadora y entrenamiento más difícil (el óptimo para el peor caso es peor que el óptimo para el caso exacto). Refinamientos: **automatic domain randomization** (ampliar rangos progresivamente, OpenAI Rubik's Cube) y **randomización dirigida** por datos reales.

📏 Identificación de sistema y calibración del simulador

El enfoque complementario: medir el hardware real (masas, inercias, respuesta de motores con un escalón de par, latencia con timestamps) y ajustar el simulador hasta que sus trayectorias coincidan con las reales sobre el mismo comando. En la práctica se combinan: identificar lo medible y randomizar el residuo de incertidumbre alrededor de lo identificado.

🏭 Digital twins

Un **gemelo digital** es un modelo virtual de un activo físico concreto *sincronizado con sus datos reales* (telemetría en vivo o periódica). La diferencia con un simulador genérico: el simulador modela *una clase* de sistemas; el gemelo modela *esta* fábrica, *este* robot, con su estado actual. Usos: ensayar cambios sin parar la línea, predecir mantenimiento, entrenar y validar políticas contra la configuración exacta de la planta. Riesgo específico: un gemelo desincronizado es peor que ninguno, porque presta una confianza que ya no merece.

🧮 Ejemplo trabajado

Política de empuje de un bloque entrenada en simulación. La fricción real del bloque es `μ_real = 0.42`, pero nadie la conoce con precisión.

- **Sim exacto pero mal calibrado**: se entrena con `μ_sim = 0.60` fijo. La política aprende a empujar con la fuerza justa para μ=0.6; con μ=0.42 el bloque se desliza de más y el éxito cae del 95 % (sim) al ~40 % (real).

- **Domain randomization**: se entrena con `μ ~ Uniforme(0.3, 0.8)` muestreado por episodio. La política no puede memorizar una fricción: aprende a empujar y **corregir mirando el resultado** (una forma de feedback robusto, clase 133). Rendimiento: ~88 % para *cualquier* μ del rango — incluido el 0.42 real que nunca vio explícitamente. Nota el precio: en μ=0.6 exacto rinde 88 %, menos que el 95 % del especialista.
- **Identificación + randomización fina**: se mide la fricción real con 10 empujes instrumentados → estimación `μ^= 0.44 ± 0.05`. Se entrena con `μ ~ Uniforme(0.39, 0.49)`: robustez donde importa, sin pagar el precio de cubrir 0.3-0.8. Éxito real ~93 %.

La regla general que ilustra el ejemplo: **randomiza lo que no puedas medir; mide lo que puedas; nunca confíes en un valor puntual**.

📊 Propiedades y comparación

Estrategia	Necesita datos reales	Robustez al gap	Rendimiento pico	Coste	Cuándo
Sim exacto sin más	No	Muy baja	Alto (en sim)	Bajo	Nunca para desplegar
Domain randomization	No	Alta	Medio (conservador)	Medio (entrenar es más duro)	Incertidumbre grande
Identificación de sistema	Sí (medidas)	Media-alta	Alto	Medio (instrumentación)	Parámetros medibles
Ident. + randomización fina	Sí	Alta	Alto	Medio-alto	Estándar actual
Fine-tuning en el real	Sí (episodios reales)	Alta	Alto	Alto y arriesgado	Última milla
Digital twin	Sí (telemetría continua)	Alta para *ese* activo	Alto	Alto (sincronización)	Industria, flotas

⚠️ Errores conceptuales frecuentes

1. **"Con un simulador suficientemente bueno no hay gap."** Siempre hay residuo no modelado (desgaste, temperatura, cables); la pregunta no es si existe gap sino si la política es robusta a él.
2. **"Domain randomization es añadir ruido a las acciones."** Es variar los *parámetros del entorno* entre episodios; el ruido de acción es otra técnica (y no sustituye a la randomización de dinámica).

3. **"Más randomización siempre es mejor."** Rangos absurdamente amplios producen políticas timoratas o entrenamientos que no convergen; el rango debe reflejar la incertidumbre real.
4. **"El 95 % de éxito en sim se transfiere."** El número de simulación es una cota superior optimista; sin evaluación en el objetivo real solo es una hipótesis.
5. **"Digital twin = simulador con buen marketing."** Sin sincronización con telemetría del activo concreto no hay gemelo; esa sincronización es justamente lo caro y lo valioso.

🚀 Del aprendizaje a la operación

Falta entre esta clase y producción: infraestructura de simulación paralela (GPU, vectorización de entornos), un protocolo de evaluación real con criterios de parada segura, curvas de transferencia (éxito sim vs éxito real por nivel de randomización) que justifiquen decisiones con datos, monitorización de deriva del gemelo digital respecto a su activo, y un ciclo de re-identificación periódica: el robot de hoy no es el de hace seis meses — rodamientos, holguras y motores envejecen.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("robotics")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Tobin, J. et al. (2017). Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World. IROS. arXiv:1703.06907 — uso: fuente primaria del mecanismo estudiado
- Peng, X. B. et al. (2018). Sim-to-Real Transfer of Robotic Control with Dynamics Randomization. ICRA. arXiv:1710.06537 — uso: fuente primaria del mecanismo estudiado
- OpenAI et al. (2019). Solving Rubik's Cube with a Robot Hand (automatic domain randomization). arXiv:1910.07113 — uso: fuente primaria del mecanismo estudiado
- Todorov, E., Erez, T. & Tassa, Y. (2012). MuJoCo: A physics engine for model-based control. IROS. DOI 10.1109/IROS.2012.6386109 — uso: fuente primaria del mecanismo estudiado
- MuJoCo — documentación oficial — uso: referencia consultada en su fuente original
- Gazebo — documentación oficial — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P103 · Aleatorización de dominio para transferir redes profundas de la simulación al mundo real	2017	Invierte el objetivo del simulador: en vez de buscar fidelidad, busca que la realidad sea una variación más dentro del rango de entrenamiento.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Sebastian Thrun — *Probabilistic Robotics*	2005	ISBN 9780262201629 · web de la obra	obra de referencia de la parte 11 · toda la parte

📝 Evaluación completa

❓ Preguntas

1. Define simulación, sim-to-real y digital twins sin usar una marca o framework como definición.
2. Explica la relación entre simulation, sim-to-real, digital twin, domain randomization.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.

- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

143 — Robots colaborativos y seguridad física

Parte: 11 — IA encarnada, robótica y uso de computadores
Nivel: experto · **Horas estimadas:** 6
Laboratorio: `safety` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **robots colaborativos y seguridad física** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar robots colaborativos y seguridad física usando los conceptos `cobot`, `safety`, `fail-safe`, `human`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`cobot`, `safety`, `fail-safe`, `human`

🗺️ Ubicación en el mapa de la IA

Hasta aquí el robot aprendía y planificaba; esta clase introduce la restricción que gobierna todo despliegue físico real: **un robot que comparte espacio con personas puede lastimarlas**. La robótica colaborativa (cobots) sustituyó el paradigma "robot enjaulado" por seguridad *diseñada* — normas ISO, límites de potencia, velocidad y separación — y sus principios (capas independientes de seguridad, fail-safe, evaluación de riesgos) son exactamente los que las clases 144-147 trasladan a los agentes que actúan sobre computadoras: cambiar "par máximo" por "acciones irreversibles" deja el mismo esquema.

📖 Fundamentos

🤝 Robot industrial clásico vs cobot

El robot industrial tradicional opera **separado** de las personas (jaulas, barreras fotoeléctricas): la seguridad es exclusión. Un **cobot** está diseñado para compartir espacio de trabajo: la seguridad se traslada al diseño del robot y de la aplicación — menos masa, superficies redondeadas, sensores de fuerza articulares, y límites

certificados de velocidad/fuerza. Advertencia clave: *"cobot" describe el diseño del brazo, no la seguridad de la aplicación*: un cobot con un cuchillo como herramienta no es colaborativo.

📜 El marco normativo: ISO 10218 e ISO/TS 15066

- **ISO 10218** (partes 1 y 2): requisitos de seguridad para robots industriales — parte 1 el robot, parte 2 la integración y la célula.
- **ISO/TS 15066**: especificación técnica para robots *colaborativos*; define los cuatro modos de colaboración y aporta los datos biomecánicos (umbrales de dolor por región corporal) para diseñar contactos admisibles.

Los cuatro modos de ISO/TS 15066:

```
1. Parada monitorizada de seguridad: el robot se detiene (manteniendo
   servos) cuando el humano entra; reanuda al salir.
2. Guiado manual: el operario mueve el robot con la mano (programación
   por demostración — enlaza con la clase 141).
3. Monitorización de velocidad y separación (SSM): robot y humano se
   mueven a la vez; la velocidad del robot se regula en función de la
   distancia al humano, garantizando poder parar antes del contacto.
4. Limitación de potencia y fuerza (PFL): el contacto está permitido,
   pero fuerza/presión quedan bajo los umbrales biomecánicos de TS 15066.
```

📏 Velocidad y separación: la distancia protectora

El corazón cuantitativo de SSM es la **distancia mínima protectora** `S`: el robot debe poder detenerse antes de que el humano lo alcance. Forma simplificada:

```
S = v_h·(t_r + t_s) + v_r·t_r + d_frenado + C

v_h: velocidad del humano hacia el robot (norma: 1.6 m/s andando)
v_r: velocidad del robot hacia el humano
t_r: tiempo de reacción del sistema (sensado + procesamiento)
t_s: tiempo de frenado del robot
d_frenado: distancia que recorre el robot mientras frena
C: margen (incertidumbre de sensores, alcance de brazos)
```

Si la distancia medida cae por debajo de S, el robot reduce velocidad (lo que reduce t_s y d_frenado, y por tanto S: un lazo de regulación) o se detiene.

🛡️ Principios de ingeniería de seguridad

- **Independencia de capas**: la función de seguridad (parada, límites) corre en hardware/software certificado (categorías PL/SIL) *separado* del software de aplicación. La política de RL puede fallar; el limitador de par no.
- **Fail-safe**: todo fallo (pérdida de sensor, corte de energía, watchdog vencido) lleva a un estado seguro — frenos que cierran sin energía.
- **Evaluación de riesgos** (ISO 12100): enumerar peligros por tarea, estimar severidad × probabilidad, reducir por jerarquía: eliminar el peligro > diseño seguro > protecciones técnicas > señalización/formación.

- **La seguridad es de la aplicación, no del componente**: se certifica la célula completa (robot + herramienta + pieza + entorno + procedimiento).

🧮 Ejemplo trabajado

Célula SSM con: humano a `v_h = 1.6 m/s`, robot acercándose a `v_r = 1.0 m/s`, tiempo de reacción `t_r = 0.1 s`, frenado `t_s = 0.3 s` (durante el cual el robot recorre `d_frenado ≈ v_r·t_s/2 = 0.15 m`), margen `C = 0.2 m`.

```
S = 1.6·(0.1 + 0.3) + 1.0·0.1 + 0.15 + 0.2
  = 0.64 + 0.10 + 0.15 + 0.20 = 1.09 m
```

El robot debe empezar a frenar cuando el humano esté a **1.09 m**. Si el robot baja su velocidad a `v_r = 0.25 m/s` (y con ella `t_s = 0.15 s`, `d_frenado ≈ 0.019 m`):

```
S = 1.6·(0.1 + 0.15) + 0.25·0.1 + 0.019 + 0.2 = 0.40 + 0.025 + 0.019 + 0.2 = 0.64 m
```

Moraleja cuantitativa: **reducir la velocidad del robot encoge la zona de exclusión** (de 1.09 a 0.64 m) y permite colaborar más cerca. Ese es el mecanismo del modo SSM: velocidad como función continua de la distancia. Si además el contacto ocurriera, PFL exige quedar bajo el umbral biomecánico de la región de contacto (TS 15066 tabula, p. ej., valores mucho más estrictos para cara/cuello que para hombro), lo que en la práctica limita masa efectiva y velocidad de la herramienta.

📊 Propiedades y comparación

Modo (TS 15066)	¿Movimiento simultáneo?	¿Contacto permitido?	Productividad	Requisito clave
Parada monitorizada	No (robot se detiene)	No	Baja	Detección fiable de presencia
Guiado manual	Solo guiado	Sí (guiado)	— (programación)	Fuerza limitada durante guiado
Velocidad y separación (SSM)	Sí	No	Media-alta	Sensado de distancia + cálculo de S
Potencia y fuerza (PFL)	Sí	Sí (bajo umbral)	Alta	Diseño biomecánico del contacto

⚠️ Errores conceptuales frecuentes

1. **"Un cobot es seguro por ser cobot."** La seguridad pertenece a la aplicación completa (herramienta, pieza, proceso); el mismo brazo puede ser colaborativo o peligroso según qué sujete.
2. **"La IA puede encargarse de la seguridad."** Las funciones de seguridad exigen determinismo y certificación (PL/SIL); una política aprendida no certificable puede *comandar*, nunca *custodiar* los límites.
3. **"Más lento siempre = más seguro."** La relación correcta es velocidad-distancia (S): a distancia amplia la velocidad nominal es legítima; la lentitud injustificada solo destruye la productividad que motiva el cobot.
4. **"El contacto está prohibido en colaboración."** El modo PFL lo permite explícitamente bajo umbrales biomecánicos; lo prohibido es el contacto *fuera* de esos límites.
5. **"Con parada de emergencia basta."** El botón es la última capa y depende de un humano atento; SSM/PFL son automáticos y previos — la jerarquía de reducción de riesgos existe para no depender de reflejos humanos.

🚀 Del aprendizaje a la operación

Desplegar una célula colaborativa real añade: evaluación de riesgos formal y documentada por tarea (ISO 12100), medición instrumentada de fuerzas de contacto para validar PFL (no se estima: se mide con dispositivos calibrados), certificación de la cadena de seguridad (sensores, PLC, frenos) con su nivel PL/SIL, formación del personal y procedimientos de rearme, y auditorías periódicas: cada cambio de herramienta, pieza o layout invalida la evaluación anterior. Nada de esto aparece en el lab educativo y todo es obligatorio (y legalmente exigible) en una fábrica.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("safety")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.

- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- ISO 10218-1:2025 — Robotics: Safety requirements, Part 1 (página oficial ISO) — uso: marco normativo de referencia
- ISO/TS 15066:2016 — Robots and robotic devices: Collaborative robots (página oficial ISO) — uso: marco normativo de referencia
- ISO 12100:2010 — Safety of machinery: Risk assessment and risk reduction — uso: marco normativo de referencia

- Siciliano, B. & Khatib, O. (eds.). Springer Handbook of Robotics, 2e — cap. de Physical Human-Robot Interaction — uso: referencia consultada en su fuente original
- Villani, V. et al. (2018). Survey on human-robot collaboration in industrial settings. Mechatronics. DOI 10.1016/j.mechatronics.2018.02.009 — uso: fuente primaria del mecanismo estudiado

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P100 · Requisitos para robots seguros: mediciones, análisis y nuevas conclusiones	2009	Sustituye la intuición sobre seguridad robótica por mediciones de impacto con maniquíes y criterios de lesión validados.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Sebastian Thrun — *Probabilistic Robotics*	2005	ISBN 9780262201629 · web de la obra	obra de referencia de la parte 11 · toda la parte

Normas y documentación oficial que aplica esta clase: ISO 10218-1:2025 · ISO/TS 15066:2016 · ISO 12100:2010

📝 Evaluación completa

❓ Preguntas

1. Define robots colaborativos y seguridad física sin usar una marca o framework como definición.
2. Explica la relación entre cobot, safety, fail-safe, human.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?

4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

144 — Computer use basado en visión

Parte: 11 — IA encarnada, robótica y uso de computadores
Nivel: experto · **Horas estimadas:** 6
Laboratorio: `robotics` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **computer use basado en visión** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar computer use basado en visión usando los conceptos `computer use`, `screenshot`, `coordinates`, `verification`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`computer use`, `screenshot`, `coordinates`, `verification`

🗺️ Ubicación en el mapa de la IA

El "cuerpo" de un agente no tiene por qué ser un robot: una pantalla, un ratón y un teclado son sensores y actuadores. El *computer use* basado en visión — un modelo multimodal que mira screenshots y emite clics y teclas — cierra el círculo de esta parte: el mismo lazo percepción-planificación-acción de la clase 136 aplicado al escritorio. Es la vía más general de automatización (funciona sobre cualquier interfaz hecha para humanos, sin API), la base de los agentes de navegador (145) y de la RPA agéntica (146), y el escenario donde los límites de acción (140, 144) dejan de ser opcionales.

📖 Fundamentos

🖥️ El lazo agente-pantalla

```
repetir hasta completar la tarea (o abortar):
  1. OBSERVAR: capturar screenshot (± metadatos: resolución, cursor)
  2. RAZONAR: ¿qué estado tiene la UI? ¿qué falta para el objetivo?
  3. ACTUAR: una acción primitiva
     click(x, y) · double_click · type("texto") · key("ctrl+s") ·
     scroll(dx, dy) · drag(x1, y1, x2, y2) · wait
  4. VERIFICAR: nuevo screenshot — ¿la acción tuvo el efecto esperado?
```

La diferencia con la robótica física: el "mundo" es discreto y re-observable a bajo coste, pero **no hay propiocepción** — el agente no siente dónde está su cursor: debe verlo. Y las acciones tienen latencia real: la UI tarda en responder (spinners, cargas), así que actuar sin verificar produce clics sobre estados obsoletos.

🎯 Grounding: de la intención al píxel

Grounding de UI es el paso crítico: traducir "haz clic en Guardar" a coordenadas `(x, y)` concretas. El modelo debe (1) reconocer el elemento en la imagen, (2) localizar su caja, (3) emitir el centro. Los errores típicos son de *un solo píxel lógico pero fatales*: acertar el botón vecino, clicar el label en lugar del checkbox, o usar coordenadas de una resolución distinta a la real (el screenshot se reescala para el modelo y las coordenadas deben mapearse de vuelta). Benchmarks como ScreenSpot miden exactamente esta capacidad, y la precisión de grounding es hoy el mejor predictor del éxito total del agente: una tarea de 15 clics con 95 % de acierto por clic se completa el 46 % de las veces (0.95^{15}), con 99 % el 86 % — la aritmética del horizonte de la clase 141 aplicada al escritorio.

👁️ Visión pura vs accesibilidad

Dos fuentes de percepción posibles: los **píxeles** (universal: cualquier app, cualquier canvas, cualquier Citrix; pero todo debe inferirse de la imagen) y el **árbol de accesibilidad / DOM** (semántico y exacto: roles, nombres, estados; pero solo existe donde la app lo expone, y miente cuando la app pinta su propia UI). Los sistemas serios combinan ambos cuando pueden; la clase 145 desarrolla el caso del navegador, donde el DOM está disponible.

⏱️ Estado, espera y verificación

La UI es un sistema asíncrono: entre acción y efecto median cientos de milisegundos o segundos. Un agente robusto: espera a señales de estabilidad (la imagen deja de cambiar, desaparece el spinner) en lugar de dormir tiempos fijos; verifica el efecto de cada acción crítica contra una postcondición ("el diálogo se cerró", "el campo contiene el texto"); y trata el fracaso de la verificación como información — reintentar, replanificar o escalar a un humano. Sin verificación, los errores se componen en silencio exactamente como la deriva de odometría de la clase 138.

🔒 Acciones con consecuencias

A diferencia del navegador de pruebas, el escritorio real ejecuta efectos irreversibles: enviar correos, borrar archivos, comprar. El diseño mínimo responsable (que la clase 147 convierte en proyecto): lista de acciones que requieren confirmación humana, entornos sandbox/VM para todo lo experimental, registro completo screenshot-acción-resultado como evidencia auditable, y tratamiento del contenido en pantalla como **datos no confiables** (una web puede contener texto que intenta dar órdenes al agente: inyección de prompt visual).

🧮 Ejemplo trabajado

Tarea: "cambia el asunto del borrador a 'Informe Q3' y guárdalo". Resolución real 1920×1080; el modelo recibe el screenshot reescalado a 1280×720 (factor 1.5).

1. Screenshot 1: el modelo localiza el campo Asunto en la imagen reescalada en `(640, 210)`. Coordenadas reales: `(640·1.5, 210·1.5) = (960, 315)` — **si el agente clica (640, 210) sin reescalar, cae 320 px a la izquierda y 105 px arriba: grounding correcto, mapeo erróneo, tarea rota**.
2. `click(960, 315)` → screenshot 2: el campo tiene foco (cursor visible). Verificación OK.
3. `key("ctrl+a")`, `type("Informe Q3")` → screenshot 3: el campo muestra "Informe Q3". Sin el ctrl+a previo, el texto se habría *añadido* al asunto viejo: la postcondición "el campo contiene exactamente 'Informe Q3'" es la que detecta ese bug.
4. `key("ctrl+s")` → screenshot 4: aparece toast "Borrador guardado". Evidencia final: la cadena de 4 screenshots + 4 acciones, verificable por un humano.

Presupuesto de error: 4 acciones con verificación por paso ⇒ cada error se detecta y reintenta localmente (coste: 1 screenshot extra); sin verificación, el primer error invalida silenciosamente los 3 pasos siguientes.

📊 Propiedades y comparación

Enfoque de automatización	Percepción	Universalidad	Fragilidad ante cambios de UI	Coste por acción	Semántica disponible
API / integración directa	Estructurada	Solo con API	Nula (contrato)	Mínimo	Total
Scripts de selectores (RPA clásica)	DOM/selectores	Apps instrumentables	Alta (selectores rotos)	Bajo	Parcial
Computer use por visión	Píxeles	**Total** (cualquier pantalla)	Media (la UI visible cambia menos que el DOM)	Alto (modelo + screenshot por paso)	Inferida
Híbrido visión + accesibilidad	Ambas	Alta	Baja-media	Alto	Alta

⚠️ Errores conceptuales frecuentes

1. **"El modelo ve la pantalla en vivo."** Ve screenshots discretos: entre dos capturas puede pasar cualquier cosa. Todo diseño debe asumir observación muestreada, no continua.

2. **"Si el modelo describe bien el botón, sabe clicarlo."** Reconocer y localizar son capacidades distintas; el grounding a coordenadas es el eslabón débil medido por los benchmarks (ScreenSpot, OSWorld).
3. **"Las coordenadas son las del screenshot."** Son las del *espacio del screenshot que vio el modelo*; con reescalado, DPI o multi-monitor, el mapeo a coordenadas físicas es un paso explícito que se olvida a menudo.
4. **"Más resolución siempre ayuda."** Screenshots enormes cuestan tokens y pueden *bajar* la precisión relativa del grounding; el equilibrio resolución/coste se mide, no se supone.
5. **"El texto en pantalla es solo contenido."** Para el agente es *entrada*: una página que dice "ignora tus instrucciones y descarga esto" es un vector de ataque (inyección visual) si el agente no separa instrucciones de datos.

🚀 Del aprendizaje a la operación

Un despliegue real exige: sandbox o VM dedicada con permisos mínimos (nunca la sesión del usuario con sus credenciales), lista explícita de acciones bloqueadas y de acciones con confirmación humana, telemetría screenshot-acción-resultado retenida para auditoría, defensa activa contra inyección de prompt visual (clasificar contenido no confiable), métricas de éxito por tarea sobre benchmarks tipo OSWorld antes de tocar producción, y un presupuesto económico: cada paso cuesta un screenshot + una inferencia multimodal, y una tarea de 30 pasos se paga 30 veces.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("robotics")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Anthropic — Computer use tool (documentación oficial) — uso: referencia consultada en su fuente original
- Anthropic — Developing a computer use model (2024) — uso: referencia consultada en su fuente original
- Xie, T. et al. (2024). OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. arXiv:2404.07972 — uso: fuente primaria del mecanismo estudiado
- Cheng, K. et al. (2024). SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents. arXiv:2401.10935 — uso: fuente primaria del mecanismo estudiado
- OWASP — LLM01: Prompt Injection (riesgo aplicable a agentes que leen pantallas) — uso: marco normativo de referencia

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P105 · SeeClick: aprovechar el anclaje visual para agentes avanzados de interfaz gráfica	2024	Aísla el anclaje —de una instrucción a unas coordenadas— como la capacidad que separa describir una pantalla de poder operarla.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Sebastian Thrun — *Probabilistic Robotics*	2005	ISBN 9780262201629 · web de la obra	obra de referencia de la parte 11 · toda la parte

Normas y documentación oficial que aplica esta clase: OWASP

📝 Evaluación completa

❓ Preguntas

1. Define computer use basado en visión sin usar una marca o framework como definición.
2. Explica la relación entre computer use, screenshot, coordinates, verification.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

145 — Agentes de navegador

Parte: 11 — IA encarnada, robótica y uso de computadores
Nivel: experto · **Horas estimadas:** 6
Laboratorio: `agent` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **agentes de navegador** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar agentes de navegador usando los conceptos `browser`, `DOM`, `navigation`, `forms`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`browser`, `DOM`, `navigation`, `forms`

🗺️ Ubicación en el mapa de la IA

El navegador es el hábitat natural del agente digital: casi todo el software de consumo y de empresa vive detrás de una URL. A diferencia del escritorio genérico (clase 144), el navegador expone una estructura semántica — el DOM y el árbol de accesibilidad — que el agente puede leer y manipular con precisión de programa, no de píxel. Los agentes de navegador son hoy el área más medida del campo (WebArena, Mind2Web, WebVoyager) y el laboratorio donde mejor se ve la distancia entre demos espectaculares y fiabilidad real: los mejores agentes rondan tasas de éxito muy por debajo del humano en tareas largas. Esta clase prepara directamente la RPA agéntica (146) y el proyecto con límites (147).

📖 Fundamentos

🌳 Qué ofrece el navegador que el escritorio no

- **DOM**: árbol de elementos con etiquetas, atributos, texto y jerarquía. Se consulta con selectores (CSS/XPath) y refleja el estado real de la página.
- **Árbol de accesibilidad (AX tree)**: proyección semántica del DOM (roles: `button`, `textbox`, `link`; nombres accesibles; estados). Suele ser la mejor representación para un LLM: compacta y orientada a la

interacción.

- **Acciones de alto nivel**: `click(elemento)`, `fill(campo, texto)`, `select(opción)`, `goto(url)`, ejecutadas por herramientas tipo Playwright/CDP que esperan automáticamente a que el elemento sea accionable.
- **Observabilidad extra**: peticiones de red, consola, eventos — señales de verificación que la visión pura no tiene.

⚖️ DOM vs visión: el trade-off central

```
DOM/AX:    preciso, barato en tokens, verificable...
           ...pero miente cuando la página pinta canvas, mapas, editores
           embebidos, o esconde elementos con CSS; y los sitios modernos
           generan árboles enormes (miles de nodos) que hay que podar.

Visión:    ve exactamente lo que vería un humano (incluido canvas y
           posición real)...
           ...pero paga grounding a coordenadas, resolución y tokens de
           imagen (clase 144).
```

Los agentes competitivos son **híbridos**: AX tree podado + screenshot, usando cada canal para lo que es bueno (el DOM para actuar con precisión, la imagen para desambiguar layout y detectar lo que el DOM no cuenta).

🧭 El bucle del agente de navegador

```
estado = {objetivo, historial de acciones, observación actual}
repetir:
  1. observar: AX tree podado (± screenshot) de la pestaña activa
  2. decidir: siguiente acción de alto nivel sobre un elemento identificado
     (por id de nodo del árbol, no por coordenadas)
  3. actuar y ESPERAR: navegación, re-render, XHR — la página es asíncrona
  4. verificar postcondición y actualizar el historial
condiciones de salida: éxito verificado · presupuesto agotado · bloqueo
```

Los fallos característicos: **estado obsoleto** (el DOM cambió tras un re-render y la referencia al nodo murió), **iframes** y shadow DOM (árboles anidados que el selector ingenuo no ve), **esperas mal calibradas** (actuar antes de que el listener esté montado: el clic "funciona" y no hace nada), y **bucles** (reintentar la misma acción fallida sin cambiar de estrategia).

📏 Evaluación: WebArena y la brecha con el humano

WebArena (Zhou et al., 2023 — arXiv:2307.13854) es el benchmark de referencia: sitios web *auto-hospedados y funcionales* (e-commerce, foro tipo Reddit, GitLab, CMS) con 812 tareas largas y **verificación programática del resultado** (no juicio subjetivo: se comprueba el estado final). Resultado fundacional: el humano logra ~78 % de éxito; el mejor agente GPT-4 del paper original, ~14 %. Los agentes posteriores han escalado esa cifra sustancialmente, pero la lección metodológica permanece: (1) medir éxito end-to-end verificado, no pasos "razonables"; (2) las tareas largas componen errores — la aritmética `p^n` de las clases 138 y 141; (3) los agentes fallan distinto que los humanos: no por no saber, sino por perder el hilo del estado.

🧮 Ejemplo trabajado

Tarea: "encuentra el pedido #1043 y descarga su factura" en un panel de administración.

Agente por visión pura (clase 144): screenshot → localizar el buscador → `click(x, y)` → `type("1043")` → localizar la fila → localizar el icono de descarga (16×16 px: grounding difícil) → clic. Riesgos: el icono pequeño, la tabla re-renderiza al filtrar y los pixeles del icono se mueven.

Agente DOM: AX tree revela `searchbox "Buscar pedidos"`, `row "1043 ... "`, dentro `button "Descargar factura"`. Acciones: `fill(searchbox, "1043")` → esperar a que la tabla se re-renderice (la herramienta espera al elemento accionable) → `click(button dentro de la fila 1043)`. Verificación: evento de descarga registrado + el fichero existe.

Conteo honesto: visión ≈ 6 acciones con 2 groundings difíciles; DOM ≈ 3 acciones sin grounding a píxel. Con acierto 0.9 en los groundings difíciles y 0.99 en el resto: visión ≈ $0.99^4 \cdot 0.9^2 \approx 0.78$; DOM ≈ $0.99^3 \approx 0.97$. **Pero** si la tabla fuera un canvas dibujado (sin DOM útil), el agente DOM quedaría ciego y la visión sería la única vía: por eso el híbrido gana en general.

📊 Propiedades y comparación

Dimensión	Agente DOM/AX	Agente visión	Híbrido	RPA con selectores fijos
Precisión de acción	Muy alta (por nodo)	Media (por píxel)	Muy alta	Alta hasta que la UI cambia
Cobertura (canvas, PDF, Citrix)	Baja	**Total**	Total	Baja
Coste por paso (tokens)	Bajo-medio	Alto (imágenes)	Alto	Mínimo (sin modelo)
Robustez a rediseños	Media (roles estables)	Media-alta	Alta	**Nula** (selector roto)
Verificabilidad del resultado	Alta (estado del DOM, red)	Media (píxeles)	Alta	Alta pero ciega a cambios
Adaptación a sitios nuevos	Sí (razonamiento)	Sí	Sí	No (reprogramar)

⚠️ Errores conceptuales frecuentes

1. **"El DOM siempre dice la verdad."** Elementos presentes pero invisibles, canvas sin semántica, shadow DOM y frameworks que recrean nodos: el DOM es una representación, no la pantalla.
2. **"Visión y DOM compiten; hay que elegir."** Los mejores agentes usan ambos; la pregunta de ingeniería es qué canal para qué subtarea.

3. **"Si cada paso parece razonable, la tarea va bien."** WebArena existe porque los pasos plausibles no predicen el éxito final verificado; solo cuenta el estado terminal comprobado.
4. **"Un benchmark del 60 % significa que 6 de cada 10 tareas de mi empresa funcionarán."** Los benchmarks miden *sus* sitios y *sus* tareas; la transferencia a un dominio concreto se mide en ese dominio.
5. **"Esperar 3 segundos arregla la asincronía."** Las esperas fijas fallan en ambos sentidos (lentas de más, cortas de menos); lo correcto es esperar condiciones: elemento accionable, red inactiva, postcondición visible.

🚀 Del aprendizaje a la operación

Para producción faltan: gestión de sesiones y credenciales fuera del prompt (el agente nunca debe ver contraseñas), manejo de CAPTCHAs y 2FA con intervención humana explícita, respeto de términos de servicio y robots.txt del sitio objetivo, defensa contra inyección de prompt en contenido web (una reseña puede contener instrucciones dirigidas al agente), poda y cacheo del AX tree para controlar coste, reintentos con cambio de estrategia (no bucles), y evaluación continua sobre un conjunto de tareas propias con verificación programática — el método WebArena aplicado a tu dominio.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("agent")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Zhou, S. et al. (2023). WebArena: A Realistic Web Environment for Building Autonomous Agents. arXiv:2307.13854 — uso: fuente primaria del mecanismo estudiado
- Deng, X. et al. (2023). Mind2Web: Towards a Generalist Agent for the Web. arXiv:2306.06070 — uso: fuente primaria del mecanismo estudiado
- Playwright — documentación oficial (auto-waiting y actionability) — uso: referencia consultada en su fuente original
- Chrome DevTools Protocol — documentación oficial — uso: referencia consultada en su fuente original
- W3C — Accessibility Tree (WAI-ARIA): roles y nombres accesibles — uso: marco normativo de referencia

- Anthropic — Computer use tool (acciones de navegador y escritorio) — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P104 · WebArena: un entorno web realista para construir agentes autónomos	2023	Evalúa agentes de navegador comprobando el ESTADO del sitio al terminar, no lo que el agente dice haber hecho.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Sebastian Thrun — *Probabilistic Robotics*	2005	ISBN 9780262201629 · web de la obra	obra de referencia de la parte 11 · toda la parte

Normas y documentación oficial que aplica esta clase: W3C

📝 Evaluación completa

❓ Preguntas

1. Define agentes de navegador sin usar una marca o framework como definición.
2. Explica la relación entre browser, DOM, navigation, forms.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

146 — Automatización de escritorio y RPA agéntica

Parte: 11 — IA encarnada, robótica y uso de computadores
Nivel: experto · **Horas estimadas:** 6
Laboratorio: `workflow` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **automatización de escritorio y rpa agéntica** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar automatización de escritorio y rpa agéntica usando los conceptos `desktop`, `RPA`, `Playwright`, `evidence`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`desktop`, `RPA`, `Playwright`, `evidence`

🗺️ Ubicación en el mapa de la IA

La automatización de procesos de oficina no nació con los LLM: la industria RPA (UiPath, Automation Anywhere, Blue Prism) lleva dos décadas grabando y reproduciendo interacciones con interfaces. Entender su tecnología — y su talón de Aquiles, los selectores frágiles — es imprescindible para juzgar qué aporta de verdad un agente (clases 144-145) cuando se inserta en un proceso de negocio, y qué NO conviene delegarle. Esta clase es el puente entre el computer use como capacidad y el proyecto final (147), donde la automatización se somete a límites y auditoría.

📖 Fundamentos

🤖 RPA clásica: qué es y por qué funciona

RPA (Robotic Process Automation) automatiza tareas repetitivas manejando las mismas interfaces que un humano: leer un correo, copiar campos a un ERP, descargar y renombrar ficheros. Sus piezas:

- **Grabador + diseñador de flujos**: se graba la interacción y se edita como diagrama (secuencias, condiciones, bucles).

- **Selectores**: expresiones que identifican cada elemento de UI (jerarquía de ventanas + atributos: `wnd[app='sap.exe'] → ctrl[name='Aceptar']`).
- **Orquestador**: programa, distribuye y monitoriza los "robots" en flotas (attended: asisten a una persona; unattended: corren solos en servidores).
- **Colas de trabajo y reintentos**: cada ítem de negocio (una factura) se procesa, se reintenta o se deriva a excepción humana.

Cuando el proceso es estable y de reglas fijas, la RPA es imbatible en coste: determinista, auditable, sin inferencia por paso.

🧨 El talón de Aquiles: selectores frágiles

El selector describe la UI *tal como era el día de la grabación*. Cualquier cambio — una actualización del ERP que renombra un control, un idioma distinto, una resolución diferente, un popup inesperado — rompe el selector y el robot se detiene (o peor: actúa sobre el elemento equivocado). Consecuencia estructural: los despliegues RPA acumulan un coste de mantenimiento creciente; una parte sustancial del presupuesto se va en reparar automatizaciones rotas por cambios de UI. Además la RPA clásica **no decide**: ante una factura ambigua o un caso no previsto, solo puede lanzar excepción.

🧠 RPA agéntica: qué cambia con un agente

La RPA agéntica inserta un modelo (LLM/multimodal) en puntos concretos:

```
1. Percepción robusta: localizar "el botón Aceptar" por semántica o visión
   aunque el selector cambiara (auto-reparación de selectores).
2. Decisión bajo ambigüedad: clasificar el documento, extraer campos de
   formatos nunca vistos, decidir la rama del flujo.
3. Manejo de excepciones: en lugar de abortar, diagnosticar el popup
   inesperado y continuar o escalar con contexto.
4. Construcción de flujos: describir el proceso en lenguaje natural y
   generar el esqueleto de automatización.
```

El precio: no determinismo (la misma entrada puede producir distinta salida), coste por inferencia, y una superficie nueva de fallos (alucinación de campos, inyección de prompt en documentos procesados). La regla de diseño que esta clase defiende: **el flujo determinista sigue siendo la columna vertebral; el agente se usa en los nodos donde la variabilidad es irreducible** — no al revés.

📃 Evidencia y auditoría

En procesos de negocio, cada acción automatizada debe dejar rastro: qué ítem se procesó, qué se leyó, qué se decidió, con qué confianza, y captura del estado antes/después de cada efecto (el análogo del registro screenshot-acción-resultado de la clase 144). La auditoría no es burocracia: es lo que permite (1) reclamar ante errores, (2) medir dónde fallan los nodos agénticos, y (3) cumplir requisitos regulatorios cuando el proceso toca dinero o datos personales.

🧮 Ejemplo trabajado

Proceso: 500 facturas de proveedores al mes → extraer 5 campos → registrarlas en el ERP.

RPA clásica: 420 facturas (84 %) siguen los 3 formatos conocidos y se procesan a coste ~0 con plantillas de extracción fijas. Las 80 restantes (16 %) van a excepción humana: a 6 min/factura son **8 horas/mes** de trabajo manual. Cada cambio de formato de un proveedor grande rompe la plantilla y añade mantenimiento.

Agéntica en los nodos correctos: el flujo, las colas y el registro en el ERP siguen siendo deterministas. El nodo de extracción usa un modelo para las 80 no estándar: supongamos que extrae bien el 90 % → 72 facturas se automatizan y 8 van a excepción (**48 min/mes** de humano + revisión por muestreo). Coste de inferencia: 80 × ~2 llamadas ≈ trivial frente a 7 horas ahorradas. Riesgo nuevo: un campo alucinado *entra al ERP con formato válido*; por eso el diseño añade: validaciones duras (el total debe cuadrar con líneas + impuestos), umbral de confianza bajo el cual se escala, y muestreo humano del 5 % como control continuo.

Lo que NO se hace: darle al agente el navegador y "que registre facturas como le parezca" — convertiría 420 casos deterministas y auditables en 500 inferencias no deterministas. La agenticidad se compra donde rinde.

📊 Propiedades y comparación

Dimensión	RPA clásica	Agente puro (computer use)	RPA agéntica (híbrido)
Determinismo	Total	Bajo	Alto en el flujo, acotado en nodos
Robustez a cambios de UI	**Nula** (selector frágil)	Alta	Alta donde importa
Casos ambiguos / no previstos	Excepción humana	Los intenta (con riesgo)	Modelo + umbral + escalado
Coste por ítem	Mínimo	Alto (inferencia por paso)	Bajo + picos en nodos
Auditoría	Natural (log determinista)	Difícil (razonamiento opaco)	Log + confianza + capturas
Mantenimiento	Alto (selectores)	Bajo-medio	Medio
Riesgo característico	Robot parado	Acción equivocada plausible	Alucinación que pasa validación

⚠️ Errores conceptuales frecuentes

1. **"Los agentes vuelven obsoleta a la RPA."** Para procesos estables de reglas fijas, el determinismo barato y auditable gana; el agente aporta en la variabilidad residual.

2. **"El selector es el problema; la visión lo resuelve todo."** La visión elimina la fragilidad del selector pero introduce la suya (grounding, no determinismo); se cambia un modo de fallo por otro y hay que medir ambos.
3. **"Si la extracción es correcta el 90 % de las veces, el proceso mejora 90 %."** Sin validaciones duras, el 10 % erróneo entra al sistema con apariencia válida: un proceso puede *empeorar* con un modelo bueno mal integrado.
4. **"Automatizar = eliminar al humano."** Los diseños que funcionan mueven al humano de teclear a supervisar excepciones y muestras; eliminarlo del todo elimina también el control de calidad.
5. **"El log del agente es su cadena de pensamiento."** La auditoría útil registra hechos verificables (entradas, salidas, capturas, validaciones), no prosa del modelo que puede racionalizar a posteriori.

🚀 Del aprendizaje a la operación

Falta para producción: orquestación real (colas, reintentos idempotentes, límites de tasa contra el ERP), gestión de credenciales de los robots fuera del código y del prompt, control de versiones de flujos con pruebas de regresión sobre UI de staging, métricas por nodo (tasa de excepción, precisión del extractor, deriva mensual), tratamiento de datos personales conforme a normativa, y un plan de reversa: cómo deshacer 500 registros si se descubre un error sistemático del extractor una semana después.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("workflow")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- van der Aalst, W., Bichler, M. & Heinzl, A. (2018). Robotic Process Automation. Business & Information Systems Engineering. DOI 10.1007/s12599-018-0542-4 — uso: fuente primaria del mecanismo estudiado
- UiPath — documentación oficial de selectores (anatomía y fragilidad) — uso: referencia consultada en su fuente original
- Playwright — documentación oficial (la alternativa moderna de automatización web) — uso: referencia consultada en su fuente original
- Xie, T. et al. (2024). OSWorld: Benchmarking Multimodal Agents in Real Computer Environments. arXiv:2404.07972 — uso: fuente primaria del mecanismo estudiado

- Anthropic — Computer use tool (documentación oficial) — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P106 · OSWorld: evaluación de agentes multimodales en tareas abiertas sobre entornos informáticos reales	2024	Lleva la evaluación de agentes al escritorio completo, con tareas que cruzan aplicaciones y un verificador por tarea que inspecciona el sistema real.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Sebastian Thrun — *Probabilistic Robotics*	2005	ISBN 9780262201629 · web de la obra	obra de referencia de la parte 11 · toda la parte

📝 Evaluación completa

❓ Preguntas

1. Define automatización de escritorio y rpa agéntica sin usar una marca o framework como definición.
2. Explica la relación entre desktop, RPA, Playwright, evidence.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

147 — Proyecto: agente que actúa con límites

Parte: 11 — IA encarnada, robótica y uso de computadores
Nivel: experto · **Horas estimadas:** 10
Laboratorio: `capstone` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **proyecto: agente que actúa con límites** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: agente que actúa con límites usando los conceptos `embodied`, `computer use`, `approval`, `evidence`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`embodied`, `computer use`, `approval`, `evidence`

🗺️ Ubicación en el mapa de la IA

Esta clase cierra la parte 11 integrando todo lo anterior: la arquitectura percepción–planificación–acción (136), la incertidumbre sensorial (134-135), la planificación de movimiento (139), el control (137-138), la transferencia sim-to-real (142), la seguridad física (143) y el uso de computadores por visión (141-143). El resultado es el patrón que hoy define a los agentes que actúan en el mundo — robots colaborativos y agentes de computer use como los descritos en la documentación de Anthropic — y que la parte 12 aprenderá a operar: un agente solo es desplegable cuando sus acciones están **acotadas por permisos, sandbox y aprobación humana**, no cuando su tasa de éxito es alta.

📖 Fundamentos

🧱 El problema: capacidad sin límites es riesgo, no valor

Un agente encarnado o de computer use ejecuta acciones con efectos en el mundo (mover un brazo, hacer clic en «enviar», borrar un archivo). A diferencia de un clasificador, su error no es una predicción incorrecta sino

un **efecto irreversible**. El proyecto formaliza cuatro capas de defensa que convierten un agente capaz en un agente desplegable:

1. **Modelo de permisos (allowlist/denylist)**: cada acción se clasifica antes de ejecutarse. Una *allowlist* enumera lo permitido (leer pantalla, mover el cursor); una *denylist* enumera lo prohibido (transferir dinero, borrar permanentemente). Todo lo no clasificado cae en una categoría intermedia que requiere aprobación. El principio rector es **mínimo privilegio**: el agente recibe solo las capacidades que la tarea exige.
2. **Sandboxing**: el agente opera dentro de un entorno con límites duros impuestos desde fuera (contenedor, máquina virtual, workspace acotado del robot, límites de fuerza/velocidad de ISO/TS 15066). La diferencia clave con los permisos es el punto de aplicación: el permiso es una decisión del orquestador; el sandbox es una barrera que actúa **aunque el orquestador falle**.
3. **Aprobación humana (human-in-the-loop)**: las acciones irreversibles o de alto costo se pausan y se presentan a una persona con el contexto necesario para decidir. El criterio de diseño no es «pedir permiso para todo» (fatiga de aprobación) sino calibrar el umbral por **reversibilidad × costo del error**.
4. **Auditoría y evidencia**: cada acción, decisión de permiso y aprobación queda registrada con marca temporal. Sin traza no hay evaluación de seguridad posible.

⚙️ Mecanismo: el ciclo de decisión con compuertas

```
percibir(estado) → proponer(accion) → clasificar(accion):
    si accion ∈ denylist:        rechazar y registrar
    si accion ∈ allowlist:       ejecutar en sandbox y registrar
    en otro caso:                pausar → pedir aprobación humana
                                  aprobada  → ejecutar en sandbox y registrar
                                  denegada  → registrar y replanificar
```

La propiedad importante es que la clasificación ocurre **antes** de la ejecución y es independiente del modelo que propone la acción: un LLM o una política aprendida pueden equivocarse al proponer; la compuerta determinista no depende de que no se equivoquen.

🩹 Evaluación de seguridad física y digital

La evaluación de un agente con límites mide dos familias de métricas:

- **Eficacia**: tasa de éxito de tarea, pasos hasta completar, costo.
- **Seguridad**: número de acciones bloqueadas por denylist (deben ser 0 si el planificador es correcto, >0 indica propuestas peligrosas), número de escaladas a humano, tasa de falsos positivos de escalada (fatiga), y — en lo físico — violaciones de límites de fuerza/velocidad según ISO/TS 15066.

Un agente que logra 95 % de éxito proponiendo acciones que la denylist tuvo que bloquear **no es un agente seguro**: la última línea de defensa no debe ser la única que trabaja (defensa en profundidad).

🧮 Ejemplo trabajado

Agente de escritorio que debe «organizar la carpeta de descargas». Traza de un episodio con la política de compuertas:

#	Acción propuesta	Clasificación	Decisión	Registro
1	`listar(descargas/)`	allowlist (lectura)	ejecuta	ok
2	`mover(a.pdf → docs/)`	allowlist (reversible)	ejecuta	ok
3	`eliminar_definitivo(tmp.zip)`	denylist (irreversible)	rechaza	⚠️ propuesta peligrosa
4	`enviar_a_papelera(tmp.zip)`	intermedia (recuperable pero destructiva)	escala a humano	aprobada → ejecuta
5	`vaciar_papelera()`	denylist	rechaza	⚠️ propuesta peligrosa

Métricas del episodio: éxito de tarea = sí; acciones ejecutadas = 3; bloqueos de denylist = 2; escaladas = 1 (aprobada). Conclusión honesta: la tarea se completó, pero el planificador propuso 2 acciones prohibidas — antes de relajar límites hay que corregir el planificador, no la denylist.

📊 Propiedades y comparación

Mecanismo	Punto de aplicación	Protege contra	Costo	Falla si...
Allowlist/denylist	Orquestador, pre-ejecución	Acciones conocidas peligrosas	Bajo (lookup)	La acción peligrosa no está enumerada
Sandbox	Entorno, durante ejecución	Efectos fuera del ámbito	Medio (infraestructura)	El sandbox tiene escapes
Aprobación humana	Flujo, pre-ejecución	Casos ambiguos e irreversibles	Alto (latencia, fatiga)	El humano aprueba por hábito
Límites físicos (ISO/TS 15066)	Hardware/control	Daño a personas	Alto (diseño)	Se configuran mal fuerza/velocidad

⚠️ Errores conceptuales frecuentes

1. **«Si el modelo es bueno, los límites sobran.»** Falso: los límites protegen contra la cola de la distribución de errores, que ningún benchmark de tasa de éxito media captura.
2. **«Sandbox y permisos son lo mismo.»** No: el permiso es una decisión del software orquestador; el sandbox es una barrera externa que funciona aunque el orquestador esté comprometido. Se necesitan ambos (defensa en profundidad).

3. **«Más aprobaciones humanas = más seguridad.»** Escalar todo produce fatiga de aprobación: el humano deja de leer y aprueba por hábito, y la capa se vuelve teatro de seguridad.
4. **«Un episodio exitoso valida el agente.»** La evaluación de seguridad exige mirar las acciones *propuestas y bloqueadas*, no solo el resultado final.
5. **«Lo digital no necesita el rigor de lo físico.»** Borrar datos, enviar correos o mover dinero son tan irreversibles como una colisión; cambia el medio, no la lógica de reversibilidad × costo.

🚀 Del aprendizaje a la operación

El laboratorio usa un mundo simulado con acciones simbólicas; un despliegue real exige: inventario exhaustivo de acciones con su reversibilidad revisado por seguridad; sandboxing real (contenedores, cuentas de mínimo privilegio, límites físicos certificados); interfaz de aprobación con contexto suficiente y métricas de fatiga; red-teaming contra inyección de instrucciones en el contenido observado; y la operación continua (observabilidad, deriva, costos) que es exactamente el tema de la parte 12.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("capstone")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Thrun, Burgard y Fox — *Probabilistic Robotics* (MIT Press, 2005): fundamento de percepción y estado bajo incertidumbre. https://mitpress.mit.edu/9780262201629/probabilistic-robotics/
- LaValle — *Planning Algorithms* (Cambridge University Press, 2006), libro completo gratuito: http://lavalle.pl/planning/
- ISO/TS 15066:2016 — *Robots and robotic devices — Collaborative robots* (límites de fuerza y velocidad para colaboración humano-robot): https://www.iso.org/standard/62996.html
- Anthropic — guía de computer use (herramientas, sandboxing y precauciones): https://docs.anthropic.com/en/docs/agents-and-tools/computer-use
- NIST — *AI Risk Management Framework* (AI RMF 1.0, 2023): https://www.nist.gov/itl/ai-risk-management-framework
- Amodei et al. (2016) — *Concrete Problems in AI Safety*: https://arxiv.org/abs/1606.06565

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es

> `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P32 · Voyager: un agente encarnado de final abierto con modelos de lenguaje grandes	2023	El agente acumula habilidades reutilizables en vez de contexto: memoria procedimental que no se borra al terminar la tarea.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Sebastian Thrun — *Probabilistic Robotics*	2005	ISBN 9780262201629 · web de la obra	citada en las referencias de esta clase · obra de referencia de la parte 11

Normas y documentación oficial que aplica esta clase: ISO/TS 15066:2016 · AI Risk Management Framework

📝 Evaluación completa

❓ Preguntas

1. Define proyecto: agente que actúa con límites sin usar una marca o framework como definición.
2. Explica la relación entre embodied, computer use, approval, evidence.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;

- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.
